Module I: Datasets and benchmarks

00 **Opening: Multimodal datasets**

Distinctive observations, annotations, and dataset contributions

01 **OctoNet**

Broad multimodal coverage with MoCap supervision

02 **RF-Behavior**

Gestures, activities, and sentiment with multi-angle RF sensing

03 **OctoSense**

Shared access, reusable components, and repeatable experiments

Opening: Multimodal dataset landscape

	Dataset	Modalities	Subjects	Label coverage	Reference data	Task focus	Year
Vision	**NTU RGB+D**	RGB-D	40	60	RGB-D pose	Activity recognition	2016
RF	**Widar3.0**	Wi-Fi	17	22	—	Gesture recognition	2019
Multimodal	**mRI**	RGB-D / mmWave IMU	20	12	Camera-derived 3D pose	Action detection / 3D pose estimation	2022
Multimodal	**OPERAnet**	Wi-Fi CSI / passive radar UWB / Kinect	6	6	Kinect skeleton	Activity recognition / passive localization	2022
Multimodal	**MM-Fi**	RGB-D / LiDAR mmWave / Wi-Fi	40	27	Camera-derived 3D pose	Action recognition / 3D pose estimation	2023
Multimodal	**XRF55**	Wi-Fi / RFID / mmWave RGB-D-IR	39	55	Kinect RGB-D-IR	Activity recognition	2024
Multimodal	**OctoNet**	12 modalities	41	62	MoCap 3D pose	Activity / 3D pose	2025
Multimodal	**CUHK-X**	7 streams	30	40	RGB-D pose	Recognition / understanding reasoning	2026
Multimodal	**RF-Behavior**	Radar / RFID / LoRa IMU / infrared MoCap	61	21 gestures 10 activities 6 sentiments	MoCap 3D pose	Gesture / activity / sentiment analysis	2026

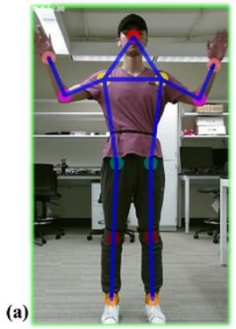
mRI: multimodal pose estimation for rehabilitation

2022 · NeurIPS (Datasets and Benchmarks) · University of Wisconsin-Madison

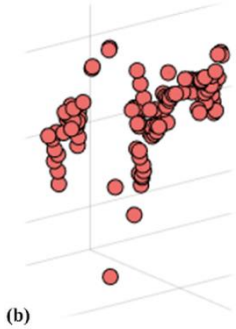
20
participants

12
movements

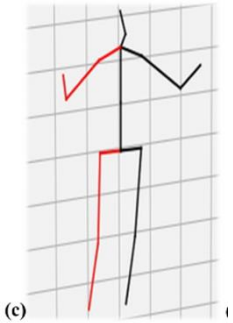
160K+
aligned frames



RGB



mmWave



3D pose reference



IMU



Depth

Rehabilitation-focused multimodal motion data

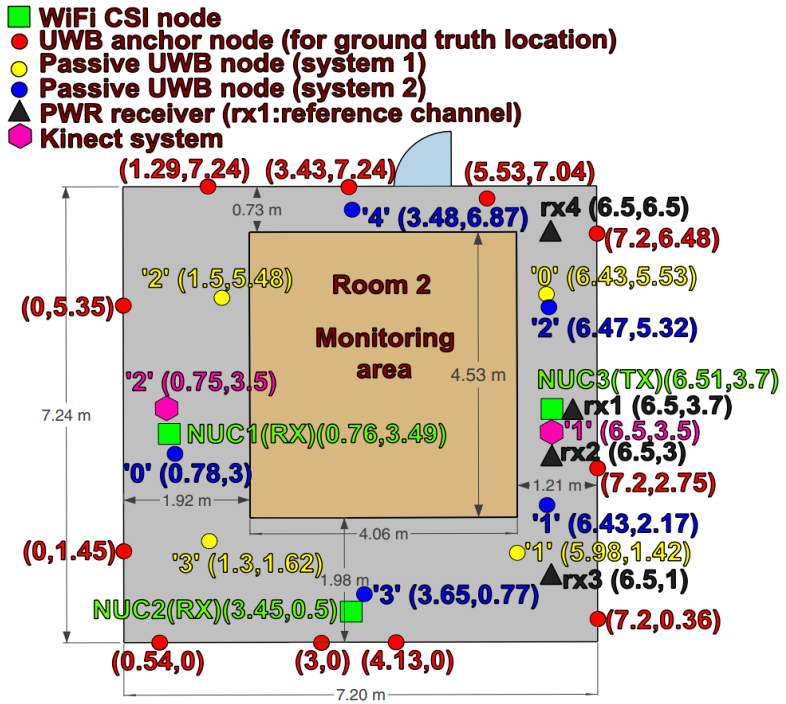
Clinically suggested rehabilitation movements pair mmWave, RGB-D and body-worn IMUs with 3D pose annotations.

OPERAnet: activity labels and location trajectories

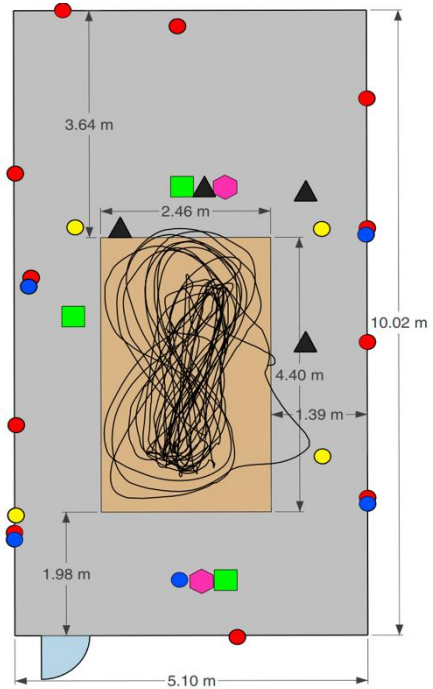
2022 · Scientific Data · University of Bristol

6	6	2	~8 h
participants	daily activities	rooms	recordings

Setup: Room 2



Localization: Room 1



Ground-truth trajectory

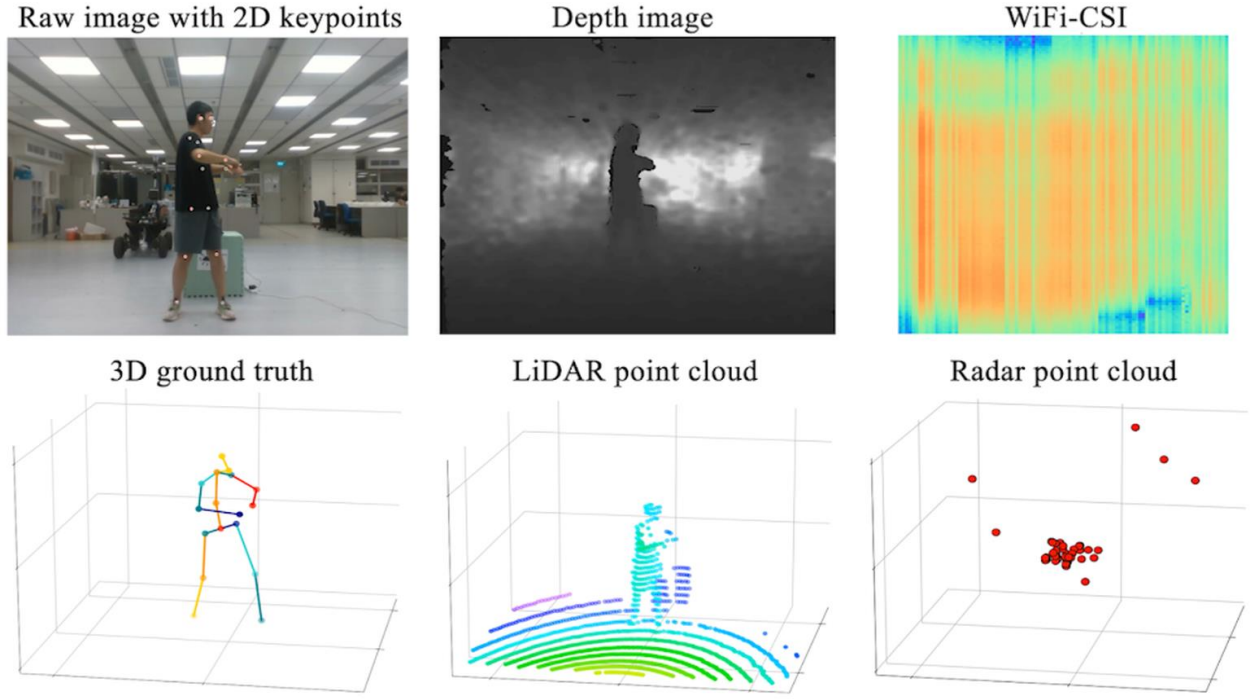
Activity and trajectory annotations for passive RF sensing

Time-resolved activity labels and UWB-tag position ground truth accompany complementary RF measurements.

MM-Fi: paired spatial sensing and body geometry

2023 · NeurIPS (Datasets and Benchmarks) · Nanyang Technological University

40 participants | **27** actions | **4** environments | **5** modalities



Paired LiDAR, radar and Wi-Fi observations

Dense LiDAR point clouds, sparse radar returns and Wi-Fi CSI are recorded alongside RGB-D.

Multi-level body geometry annotations

Annotations include 2D/3D keypoints, 3D position and image-estimated dense pose.

XRF55: multi-band RF data for indoor actions

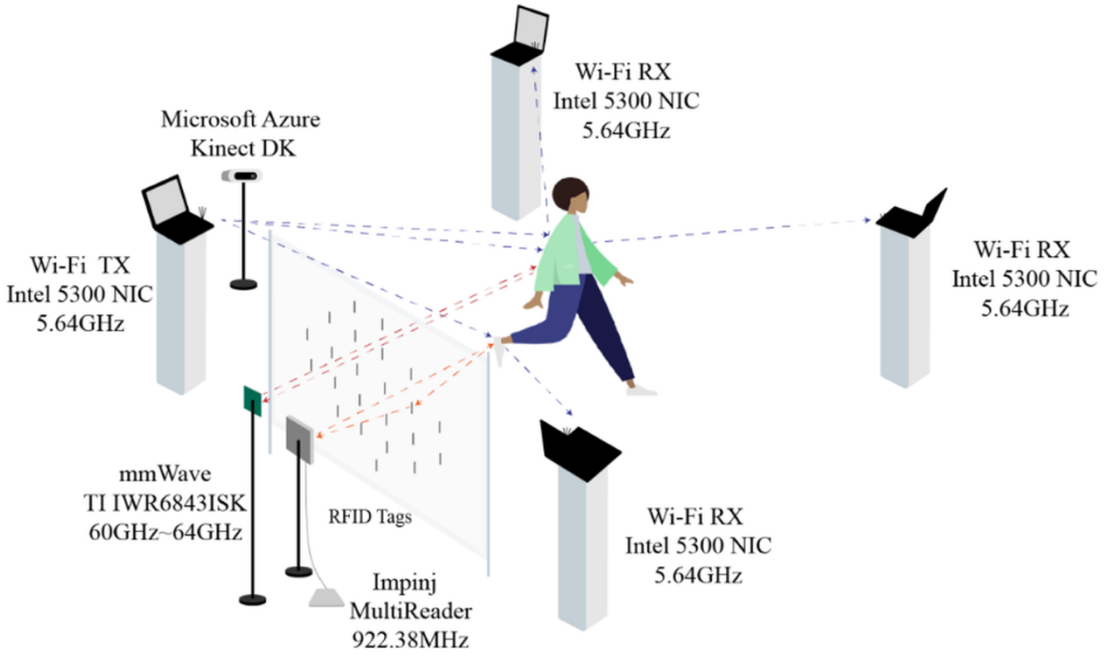
2024 · ACM IMWUT · Xi'an Jiaotong University

39
participants

55
action classes

4
scenes

42.9K
RF samples



Multi-band RF recordings across indoor actions

RFID, Wi-Fi and mmWave recordings share a 55-class, application-oriented indoor action vocabulary.

CUHK-X: sensor-caption pairs with scene context

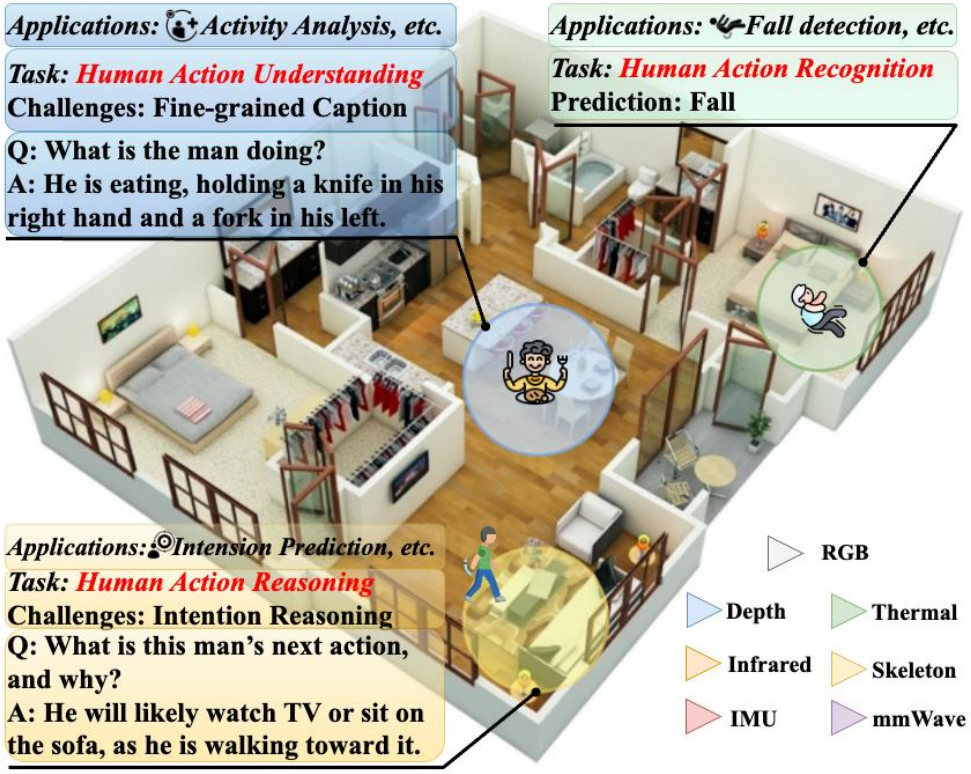
2026 · ACM MobiSys · The Chinese University of Hong Kong

30 participants

40 actions

7 streams

3 benchmarks



Sensor-caption pairs with scene context

Multimodal recordings are paired with descriptions of connected activities, preserving scene context and temporal relationships.

Human-checked, LLM-generated scene descriptions guide data collection.

RF-Behavior: RF motion data with sentiment annotations

2026 · IMWUT· Aalto University

61
participants

Up to 13
radars

3
task families



RF behavior data with sentiment annotations

Dedicated sentiment recordings include participant self-assessments alongside gesture and activity campaigns.

Ground-and-ceiling radar views

Task-specific radar configurations provide complementary horizontal and overhead observations.

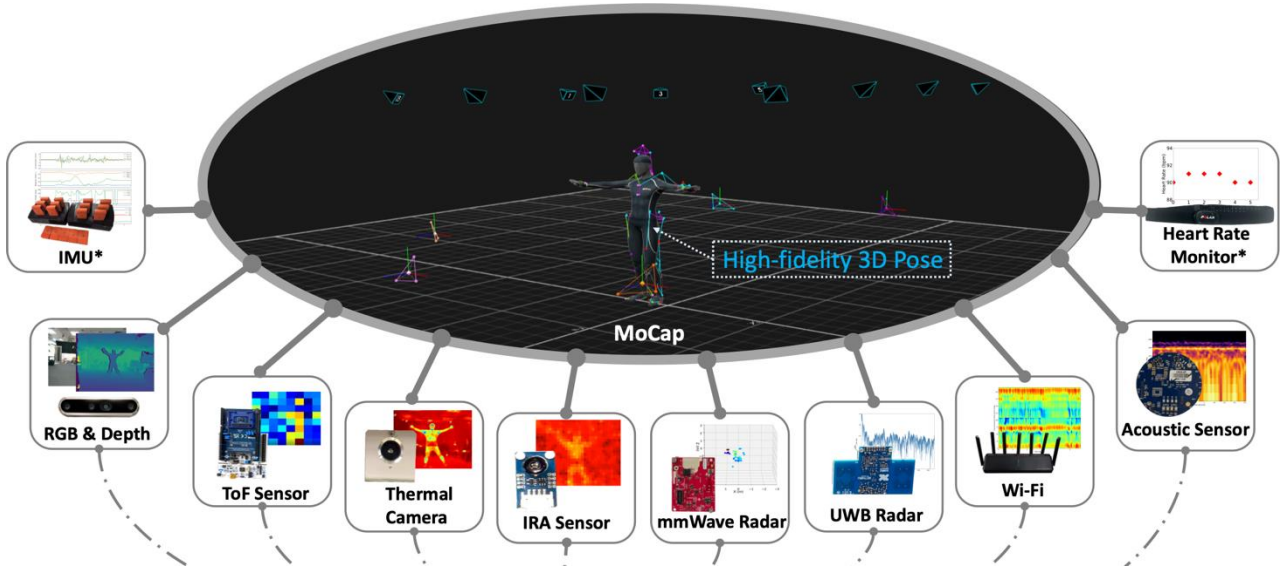
OctoNet: broad sensing coverage with MoCap labels

2025 · NeurIPS (Datasets and Benchmarks) · The University of Hong Kong

12
modalities

41
participants

62
activities



Professional MoCap supervision across 12 modalities

RF, visual, thermal, acoustic and wearable recordings share motion-captured 3D pose supervision.

Next in Module I

01



Xie Zhang

The University of Hong Kong

OctoNet

Collection and release
MoCap-grounded data

02



Si Zuo

Aalto University

RF-Behavior

Behavior and sentiment
Multi-angle RF data

03



Weiying Hou

The University of Hong Kong

OctoSense

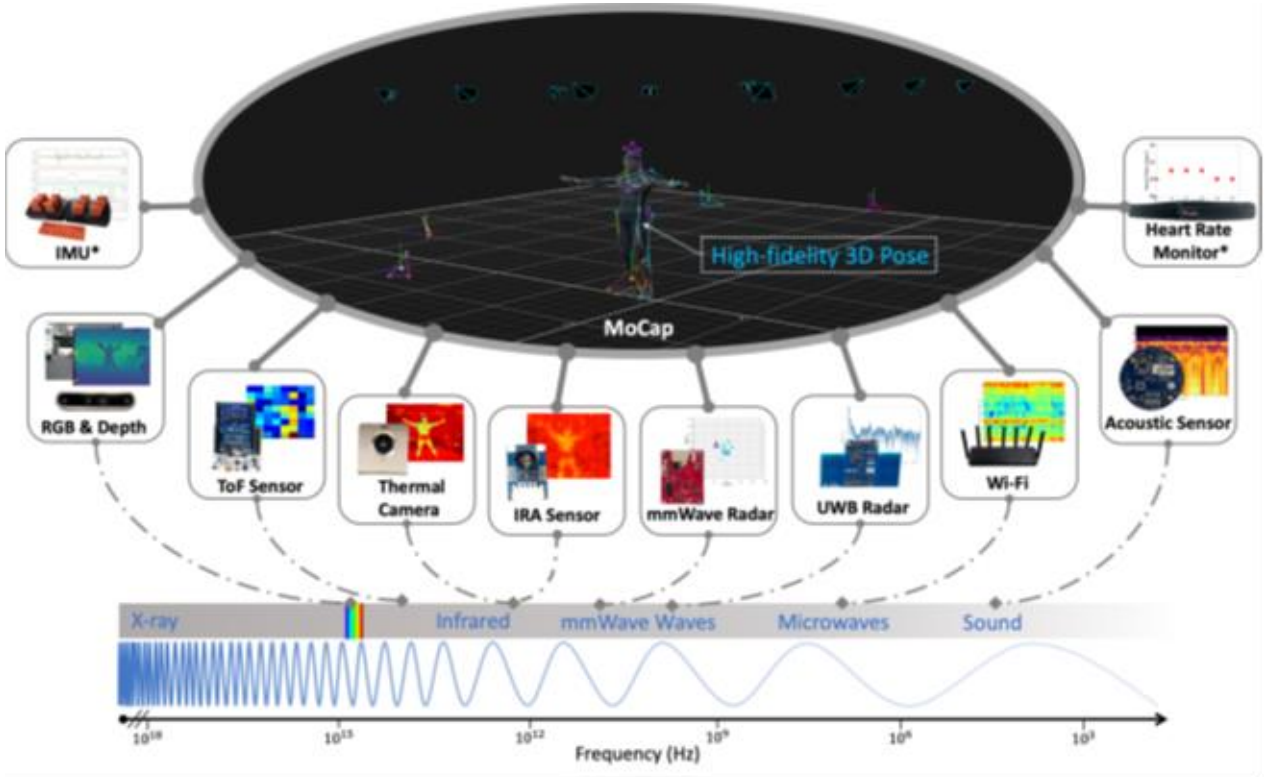
Shared data access
Reusable RF experiments

A Large-Scale **Multi-Modal** Dataset for **Human Activity Understanding** Grounded in **Motion-Captured 3D Pose** Labels

(NeurIPS 2025 Datasets and Benchmarks Track)

Dongsheng Yuan*, Xie Zhang*, Weiying Hou, Sheng Lyu, Yuemin Yu, Luca Jiang-Tao Yu, Chengxiao Li, Chenshu Wu

- Background and Motivation
- Multimodal Dataset Landscape
- OctoNet Collection
- OctoNet Release
- Future Dataset Infrastructure



OctoNet : Background and motivation

The physical world offers more than vision!

Visual



- Rich appearance and geometry information
- Oclusion, lighting, and privacy issues

RF



- Contactless & through-wall sensing
- Multipath & environment sensitivity

Thermal



- Darkness / Low-light robustness
- Spatial & temperature resolution

Acoustic



- Ambient / active sensing
- Environmental noise

Wearable



- Direct body motion
- User compliance

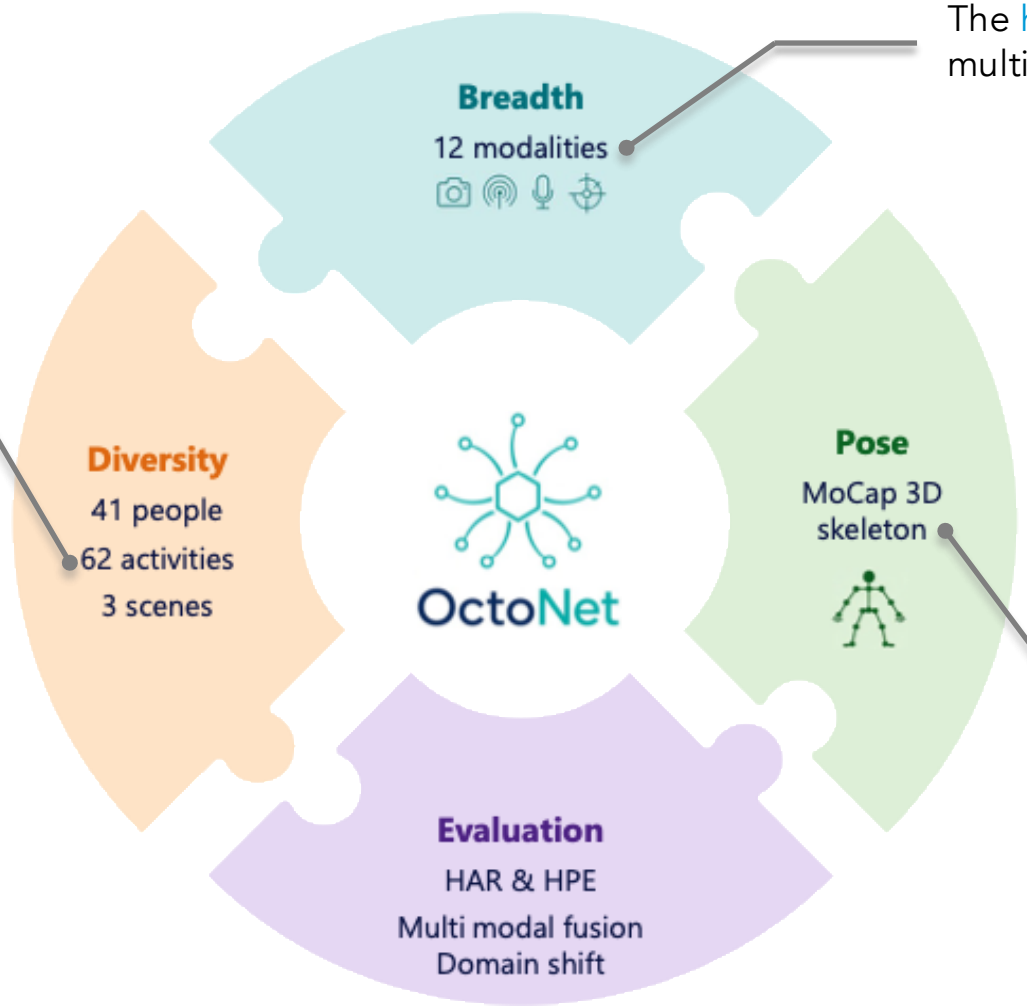
The complementary nature of these modalities drives multi-modal fusion, cross-modal learning, and sensing foundation models — **all anchored on aligned datasets**.

OctoNet: Multimodal dataset landscape

		Modalities	Subjects	Activities	Pose	Benchmark	Year
Vision	**NTU RGB+D**	RGB-D	40	60	RGB-D Pose	HAR	2016
RF	**Widar3.0**	WiFi	17	22	None	Gesture	2019
Multimodal	**mRI**	RGB-D/ mmWave/IMU	20	10	RGB-D Pose	HAR/3D pose estimation	2022
Multimodal	**OPERAnet**	RGB/UWB/ WiFi	6	6	None	HAR	2022
Multimodal	**MM-Fi**	RGB-D/ mmWave/WiFi	40	27	RGB-D Pose	HAR/3D pose estimation	2023
Multimodal	**XRF55**	RGB-D/ mmWave/WiFi	39	55	RGB-D Pose	HAR	2024
Multimodal	**OctoNet**	12 modalities	41	62	MoCap 3D Pose	HAR/3D pose estimation	2025
Multimodal	**CUHK-X**	7 modalities	30	40	RGB-D Pose	HAR/HAU/Human activity Resoning	2026
Multimodal	**RF-Behavior**	5 modalities	61	37	MoCap 3D Pose	HAR/Pose/Emotion Recognition	2026

OctoNet: Multimodal dataset landscape

OctoNet is differentiated by the combination



The highest number among existing multi-modal sensing datasets

Features 62 activity types — surpassing existing datasets in activity scope.

First dataset to link RF-sensing directly with MoCap 3D pose ground truth (prior works rely on RGB-D estimated poses)

How OctoNet was collected

Modality



Device



Place & Participant

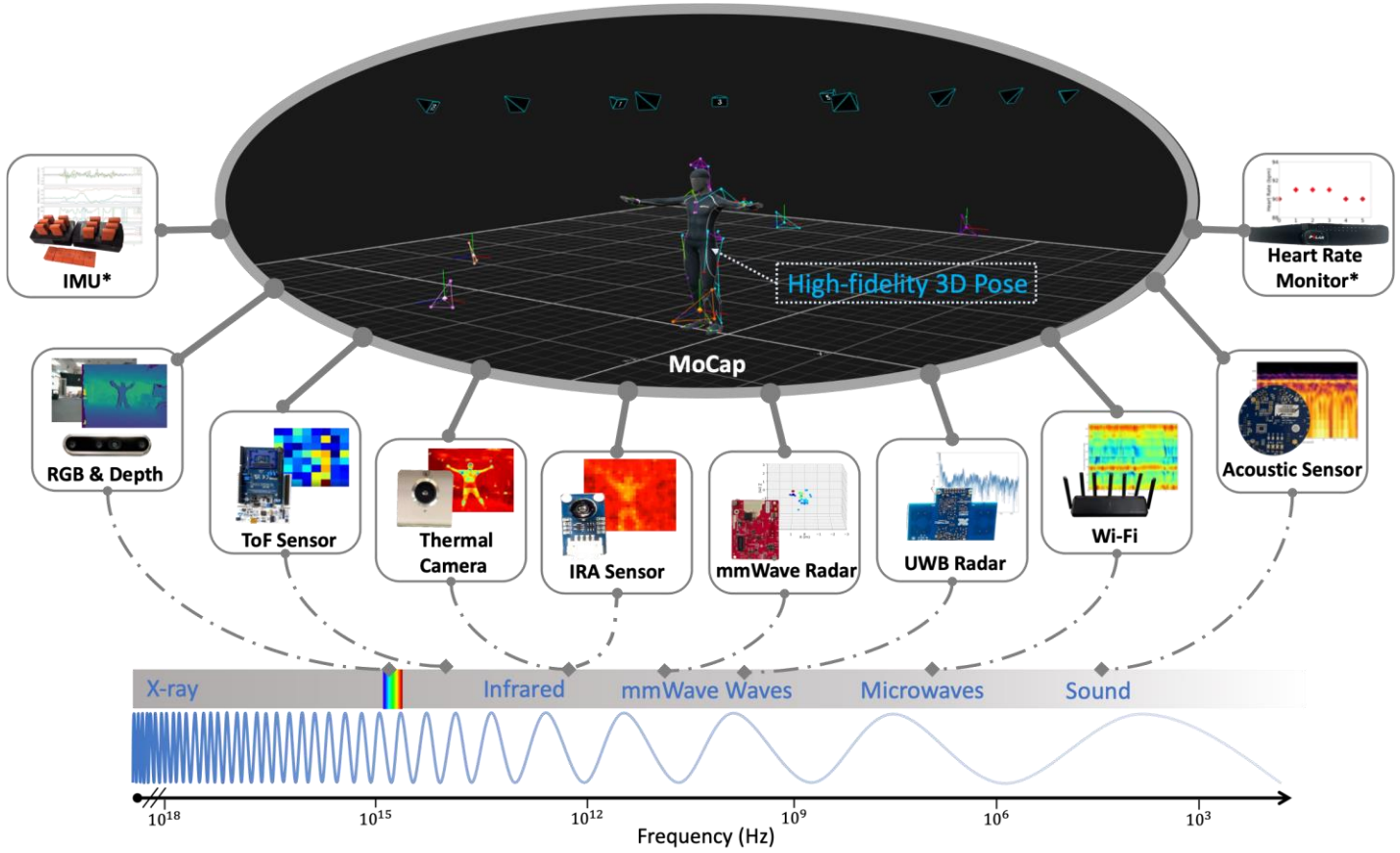


Activity



How OctoNet was collected

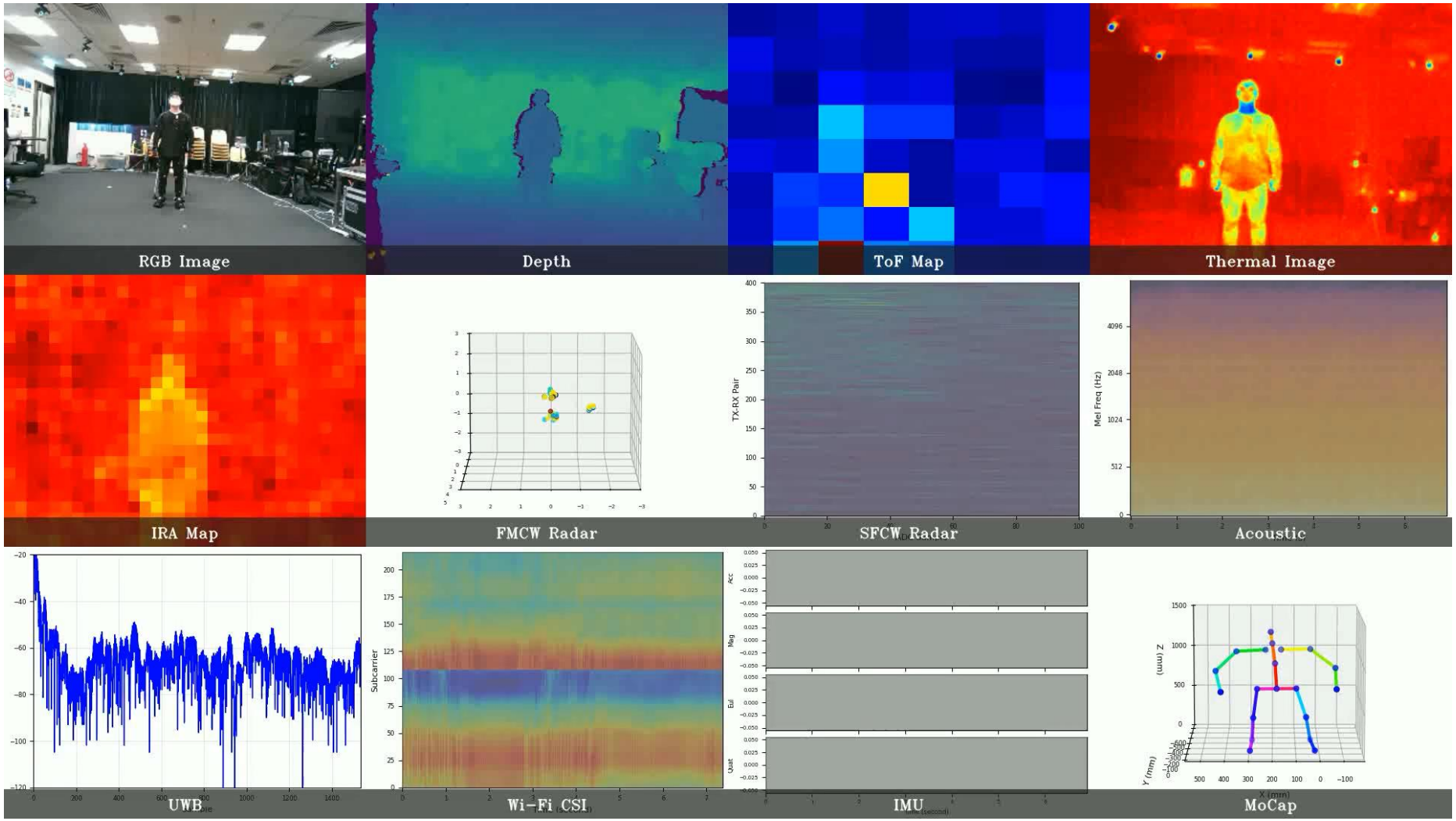
Twelve modalities span the sensing spectrum



- **Visual and thermal**
  RGB-D · ToF · Thermal · Thermal array
- **Radio frequency**
  FMCW mmWave · SFCW mmWave · UWB · Wi-Fi CSI
- **Acoustic and wearable**
  Microphones + active ultrasound · 17 IMUs · Heart rate
- **Shared supervision**
  OptiTrack motion capture

How OctoNet was collected

Heterogeneous Representations Across Modalities



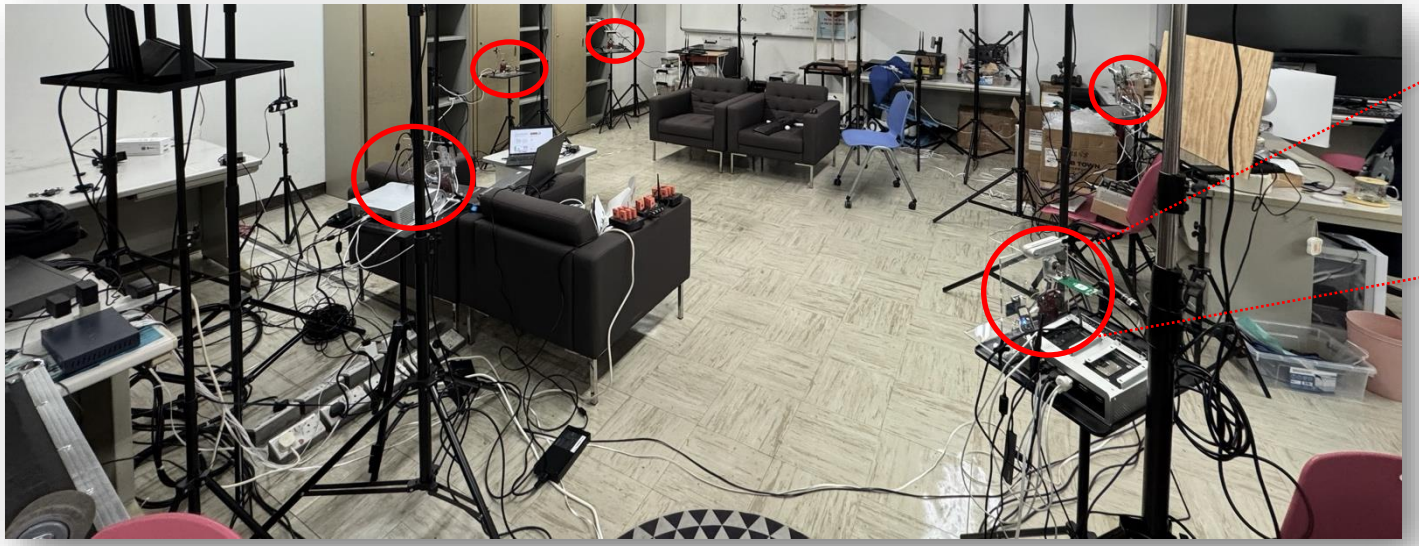
How OctoNet was collected

Multi-Modal Collocated Nodes: Every node deploys a co-located set of heterogeneous sensors.





Modality	Node-X				
	1	2	3	4	5
RGB-D	✓	-	✓	-	✓
ToF	-	-	-	✓	-
Thermal	✓	-	✓	-	-
IRA	✓	✓	✓	✓	✓
FMCW	✓	✓	✓	✓	✓
SFCW	-	-	✓	-	-
UWB	✓	-	-	-	-
Microphone	✓	-	-	✓	-
Speaker	-	-	-	✓	-
HR	✓	-	-	-	-





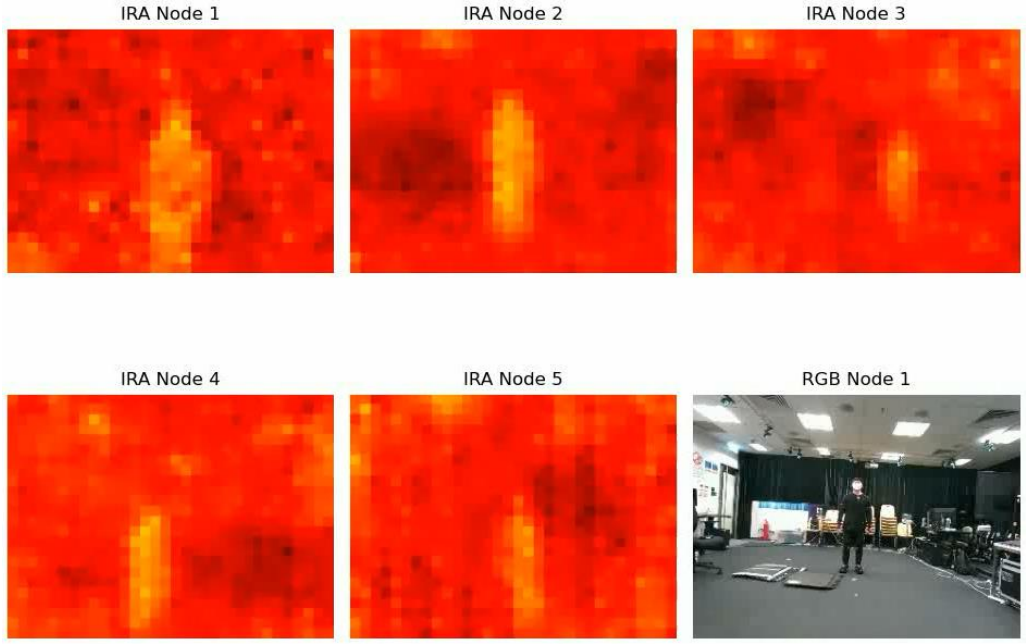
NTP Synchronization: Master mini-PC drives time alignment across all nodes.

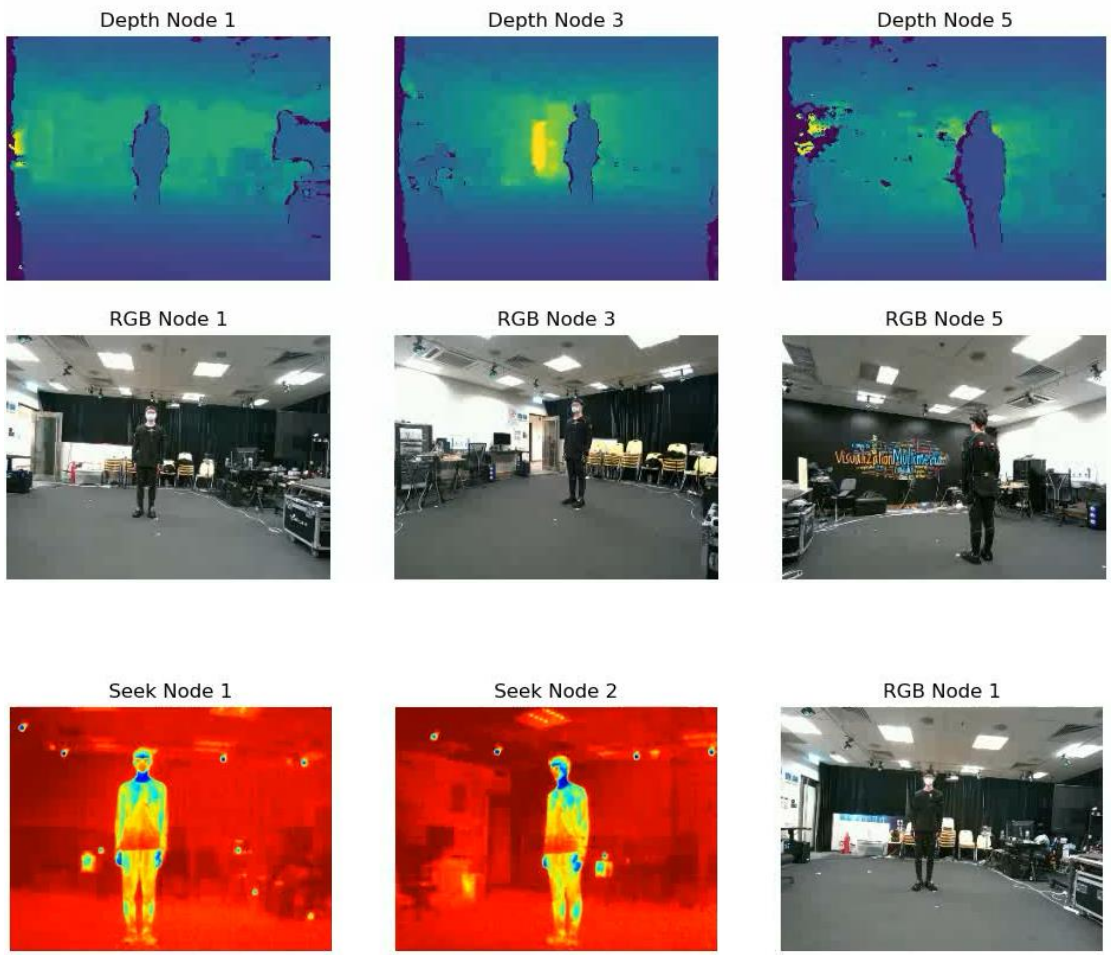
How OctoNet was collected

Five nodes create multi-view sensing

- RGB-D
- Thermal Camera
- Thermal Array Sensor
- mmWave radar
- Acoustic sensing divice (microphone)

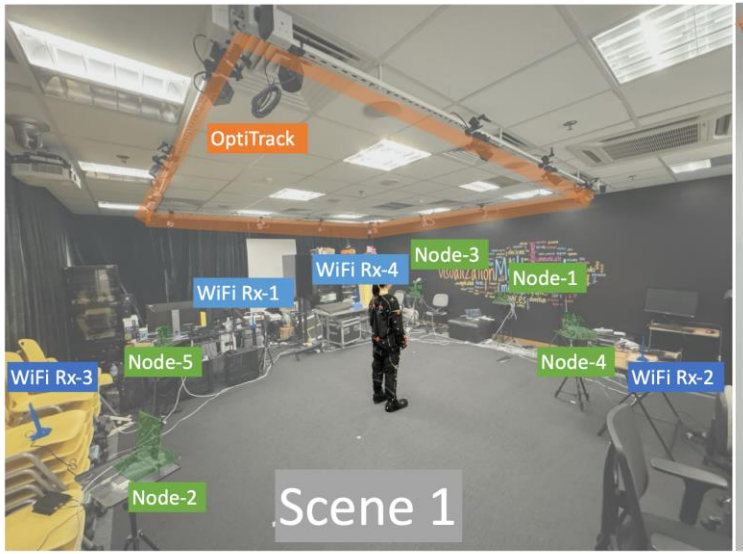
- Cross-location robustness testing
- Distributed sensor aggregation
- Cross-view contrastive learning
- View-transfer learning

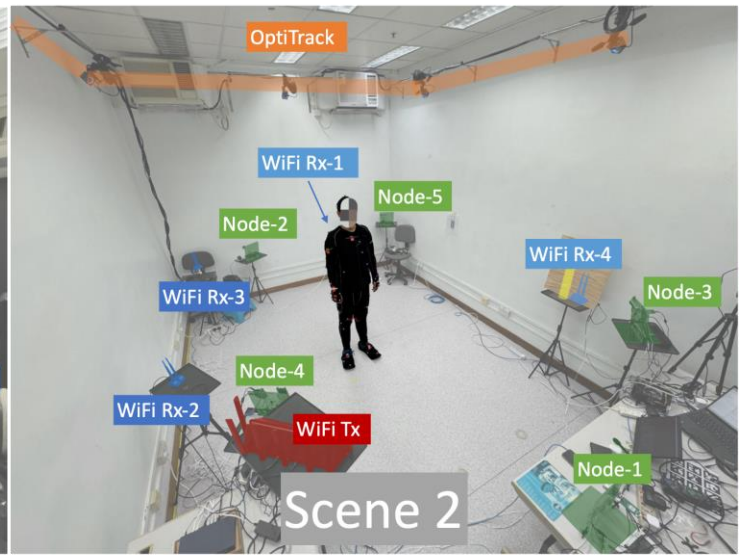


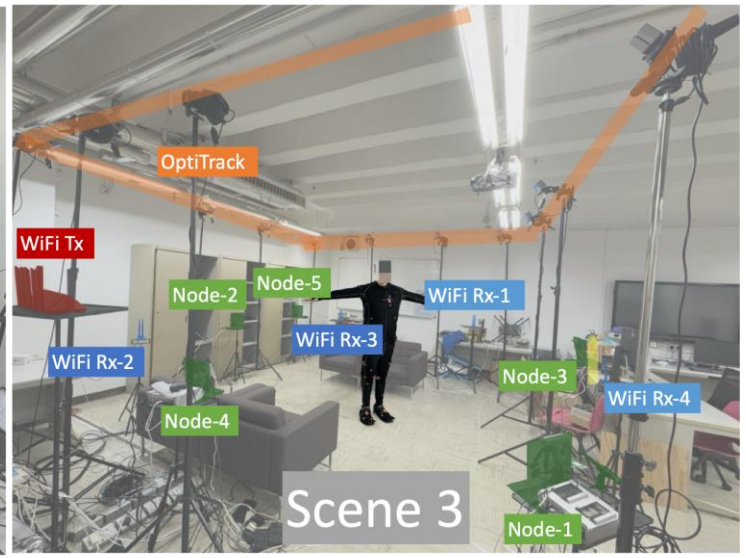


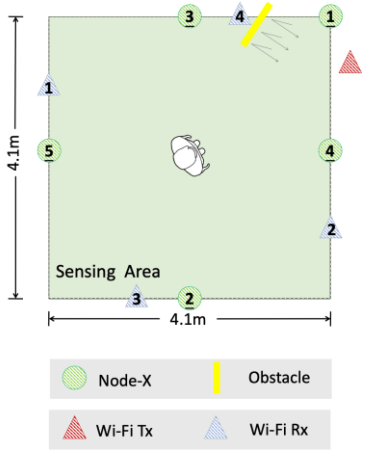
How OctoNet was collected

Three scenes test environmental transfer









Consistent Layout: All three scenes share a similar sensing node distribution.

Scene 3 is held out in the **cross-scene** benchmark.

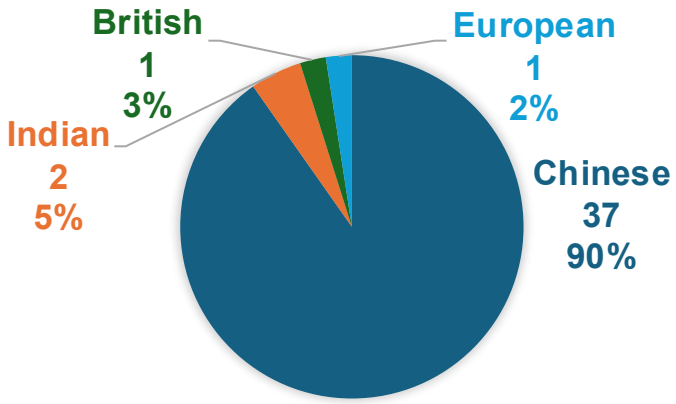
Expanding to **unrestricted, in-the-wild** environments is left to future work.

Who and what were recorded?

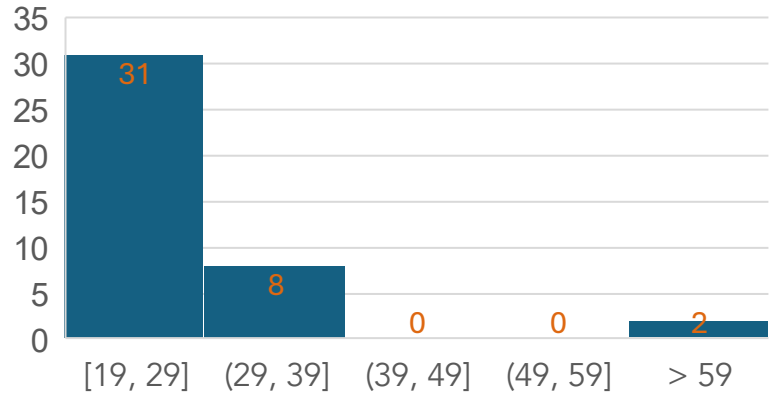
41
Participants

22
Male

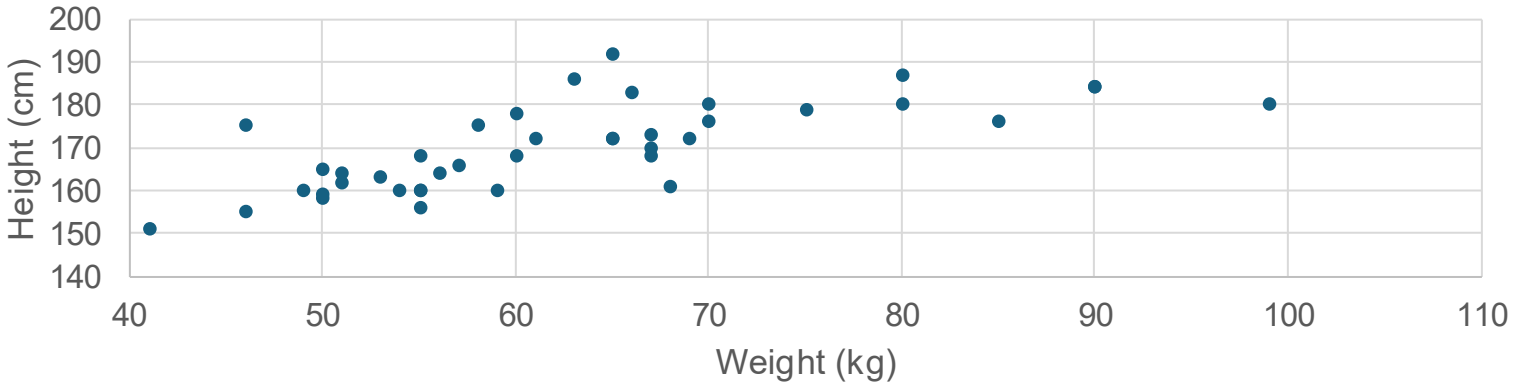
19
Female



Ethnicity of the cohort



Age-group distribution



Height versus weight

How OctoNet was collected

Who and what were recorded?



ID	Activity Name	ID	Activity Name	ID	Activity Name
1	Sitting	2	Walking	3	Bowing
4	Sleeping	5	Dancing	6	Jogging
7	Falling Down	8	Jumping	9	Jumping Jack
10	Squatting	11	Lunging	12	Turning
13	Push-Up	14	Leg Raising	15	Air Drumming
16	Boxing	17	Shaking Head	18	Answering Phone
19	Eating	20	Drinking	21	Wiping Face
22	Picking Up	23	Jumping Rope	24	Mopping Floor
25	Brushing Hair	26	Bicep Curl	27	Playing Phone
28	Brushing Teeth	29	Typing	30	Thumbs-Up
31	Thumbs-Down	32	Making OK Sign	33	Making Victory Sign
34	Drawing Circle Clockwise	35	Drawing Circle Counterclockwise	36	Stop Sign
37	Pulling Hand In	38	Pushing Hand Away	39	Handwave
40	Sweeping	41	Clapping	42	Sliding
43	Drawing Zigzag	44	Dodging	45	Bowling
46	Lifting Up A Hand	47	Tapping	48	Spreading and Pinching
49	Drawing Triangle	50	Sneezing	51	Coughing
52	Staggering	53	Yawning	54	Blowing Nose
55	Stretching Oneself	56	Touching Face	57	Shaking Hands
58	Hugging	59	Pushing Someone	60	Kicking Someone
61	Punching Someone	62	Conversation		

Body-Motion Only

Human-Object Interaction

Human-Computer Interaction

Medical Conditions

Human-Human Interaction

62 activities cover five contexts

- **Continuous Execution:** Participants perform activities continuously with ~3-second pauses between actions.
- **Repetition:** Executed 10 times per recording segment [8.76 thousand aligned activity segments].

How OctoNet was collected

Who and what were recorded?



Programmed aerobics



Freestyle

- Each participant chooses **3–5** self-selected movements
- Unstructured, spontaneous execution
- Broader coverage of personal motion style

How OctoNet was collected

Motion capture supplies the ground truth



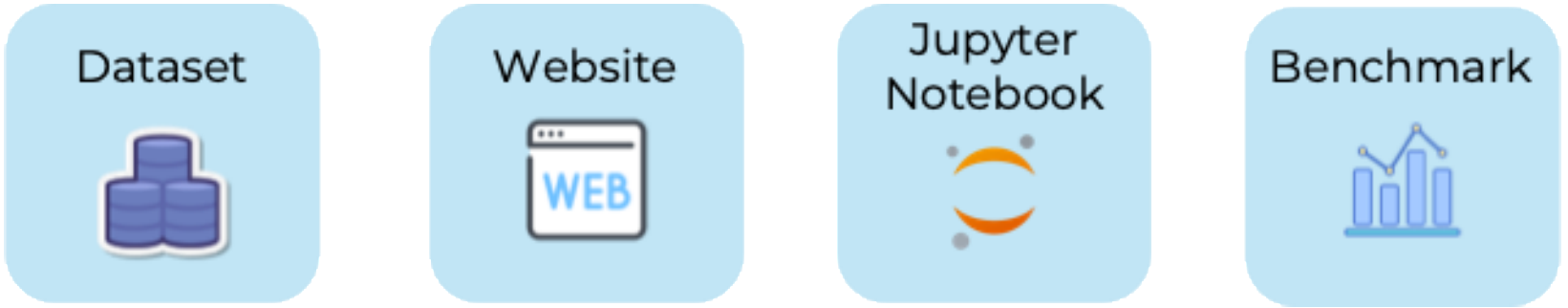
OptiTrack configuration

- 12 PrimeX 13 cameras
- 50 reflective body markers
- 120 Hz motion capture
- Manufacturer-reported position error within ~0.20 mm

Released supervision

- Full motion-capture CSV data
- A benchmark representation of 20×3 joint coordinates

What OctoNet delivers



What OctoNet delivers



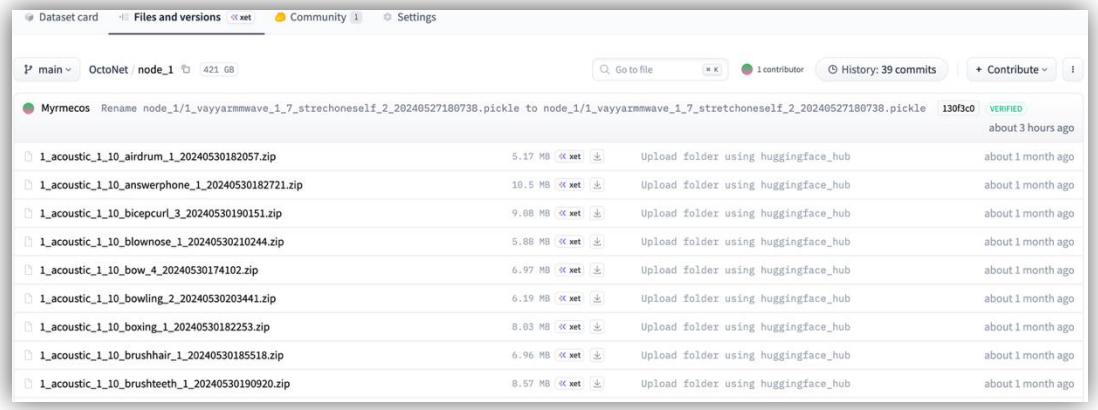
Main Dataset

https://huggingface.co/datasets/hku-aiot/OctoNet

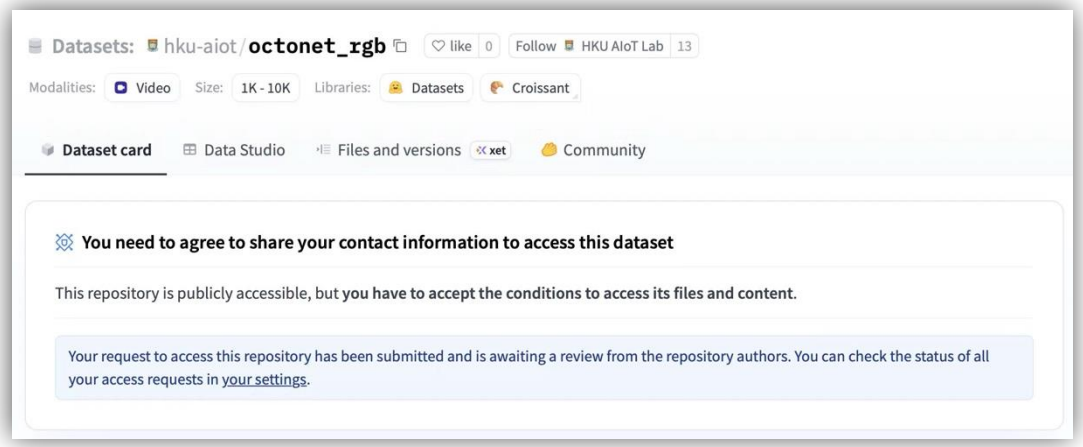
RGB Data

https://huggingface.co/datasets/hku-aiot/octonet_rgb

Hugging Face



[nodeID]_[modalityName]_[environmentID]_[userID]_[activityName]_[trialID]_[timestamp].[filetype].



- **Streamable Directory Layout:** Flat list structure designed for effortless streaming and selective downloading.

```
python streaming.py --config_path config_download.json --local_dir downloaded_data
```

- **Metadata-Rich Naming:** Self-descriptive naming schema incorporating complete file metadata.
- **Privacy & Access Control:** Sensitive modalities (e.g., RGB) are managed via Hugging Face Gated Datasets, requiring approved access requests.

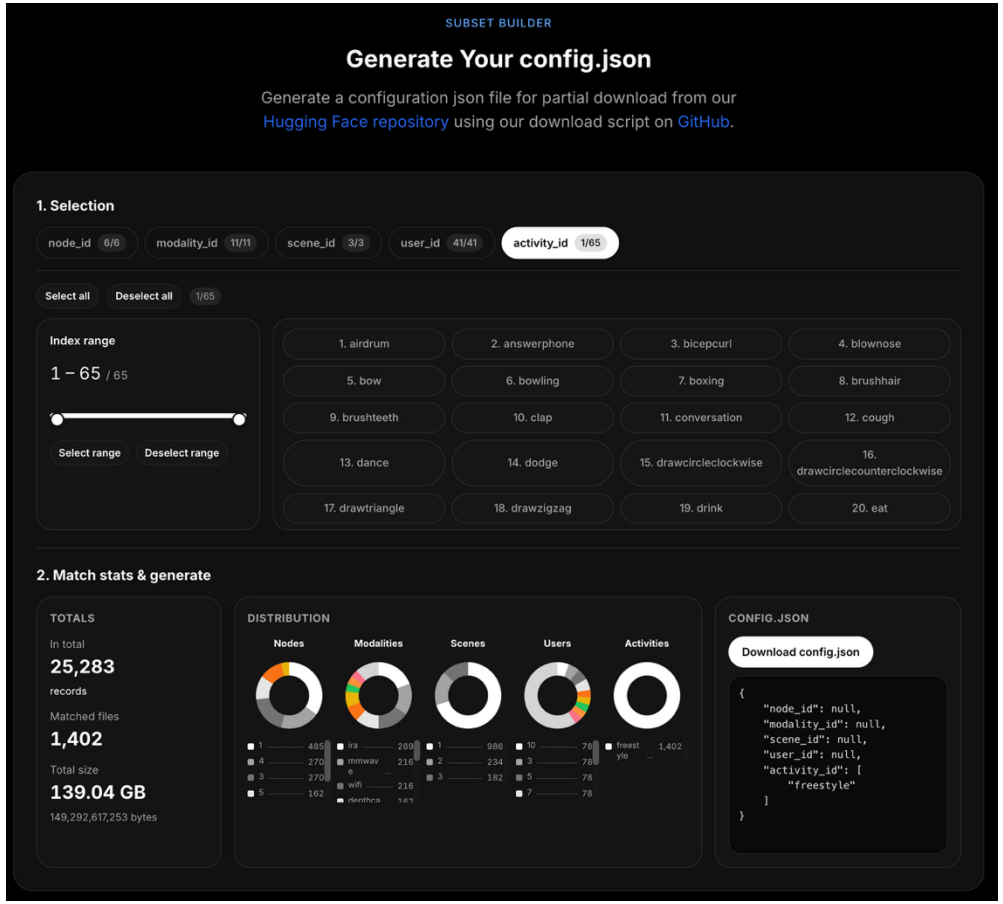
What OctoNet delivers

Dataset | Website | Jupyter Notebook | Benchmark

Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

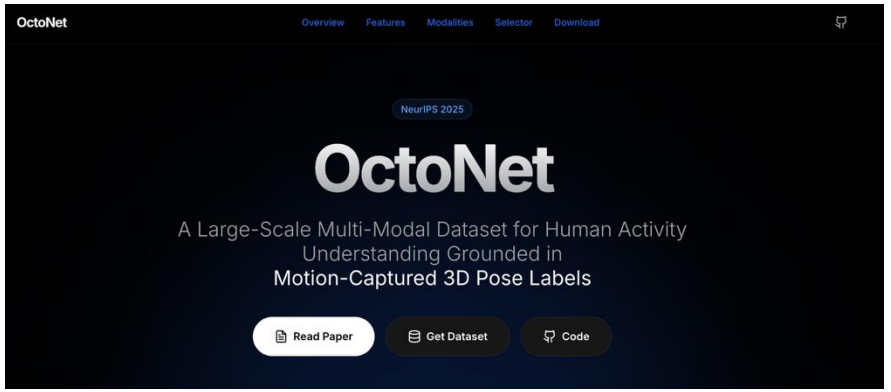
RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

Web Page
https://aiot-lab.github.io/OctoNet/



- Interactive Dataset Portal: Features dataset specifications, collection videos, and sensor setup visuals.

- Smart Data Selection Tool: Interactive web utility to filter data subsets and generate configuration files for seamless streaming downloads.



What OctoNet delivers

Dataset | Website | Jupyter Notebook | Benchmark

Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

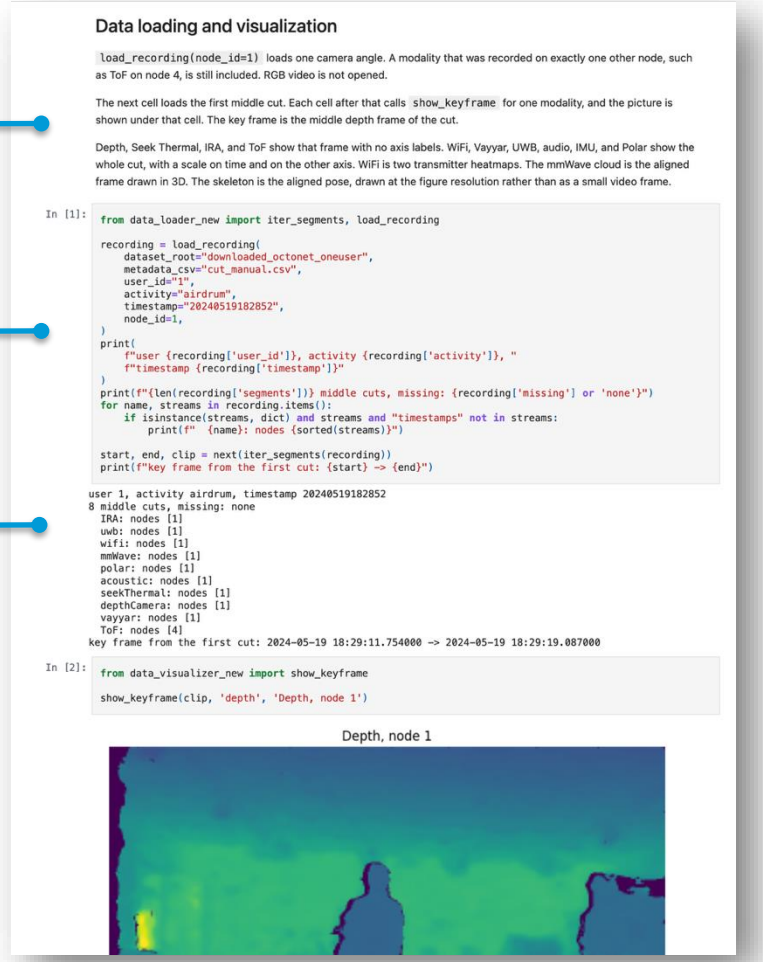
Web Page
https://aiot-lab.github.io/OctoNet/

Note-book
https://github.com/aiot-lab/OctoNet_demo

streaming.py

data_loader.py

data_visualizer.py



Data loading and visualization

`load_recording(node_id=1)` loads one camera angle. A modality that was recorded on exactly one other node, such as ToF on node 4, is still included. RGB video is not opened.

The next cell loads the first middle cut. Each cell after that calls `show_keyframe` for one modality, and the picture is shown under that cell. The key frame is the middle depth frame of the cut.

Depth, Seek Thermal, IRA, and ToF show that frame with no axis labels. WiFi, Vayyar, UWB, audio, IMU, and Polar show the whole cut, with a scale on time and on the other axis. WiFi is two transmitter heatmaps. The mmWave cloud is the aligned frame drawn in 3D. The skeleton is the aligned pose, drawn at the figure resolution rather than as a small video frame.

In [1]:
```
from data_loader_new import iter_segments, load_recording

recording = load_recording(
    dataset_root="downloaded_octonet_oneuser",
    metadata_csv="cut_manual.csv",
    user_id="1",
    activity="airdrum",
    timestamp="20240519182852",
    node_id=1,
)
print(
    f"user {recording['user_id']}, activity {recording['activity']}, "
    f"timestamp {recording['timestamp']}"
)
print(f"{len(recording['segments'])} middle cuts, missing: {recording['missing'] or 'none'}")
for name, streams in recording.items():
    if isinstance(streams, dict) and streams and "timestamps" not in streams:
        print(f"  {name}: nodes {sorted(streams)}")

start, end, clip = next(iter_segments(recording))
print(f"key frame from the first cut: {start} -> {end}")
```

```
user 1, activity airdrum, timestamp 20240519182852
8 middle cuts, missing: none
  IRA: nodes [1]
  uwb: nodes [1]
  wifi: nodes [1]
  mmWave: nodes [1]
  polar: nodes [1]
  acoustic: nodes [1]
  seekThermal: nodes [1]
  depthCamera: nodes [1]
  vayyar: nodes [1]
  ToF: nodes [4]
key frame from the first cut: 2024-05-19 18:29:11.754000 -> 2024-05-19 18:29:19.087000
```

In [2]:
```
from data_visualizer_new import show_keyframe

show_keyframe(clip, 'depth', 'Depth, node 1')
```

Depth, node 1

- **Full Multi-Modal Demos:** Executable Jupyter notebooks demonstrating streaming downloads, sample loading, and visualization across all 12 modalities.

- **Instant Quick-Start:** Out-of-the-box pipeline that abstracts away background streaming, data loading, and preprocessing.

What OctoNet delivers

Dataset | Website | Jupyter Notebook | Benchmark

Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

Web Page
https://aiot-lab.github.io/OctoNet/

Note-book
https://github.com/aiot-lab/OctoNet_demo

Bech-mark
https://github.com/aiot-lab/OctoNet

Tasks

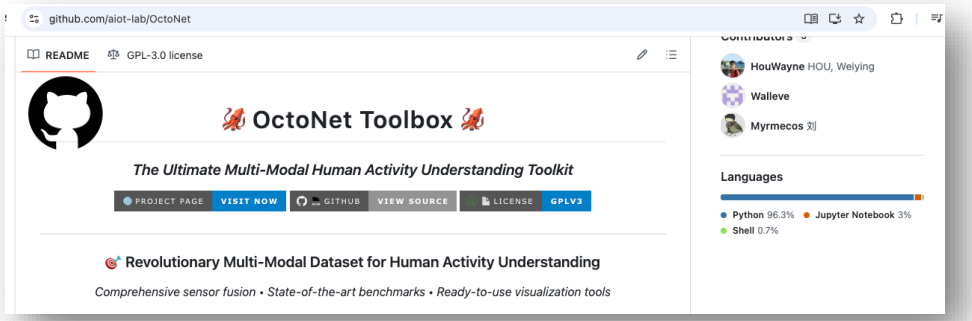
- HAR: 10-class subset and full 62-class recognition Metric: top-1 accuracy
- HPE: predict 20 joint coordinates Metric: MPJPE in millimeters

Protocols

- ID: Scene 1 + Scene 2, split 7:1:2
- CU: test on unseen users from Scene 1
- CS: test on the entirely unseen Scene 3

Baselines
ResNet · DenseNet · Swin-T · RFNet

- Comprehensive Open-Source Benchmark: Evaluates both single-modality baselines and multi-modal fusion strategies.
- Rigorous Evaluation: Standardized evaluation protocols spanning both in-domain testing and cross-domain generalization.



What OctoNet delivers

Dataset · Website · Jupyter Notebook · Benchmark

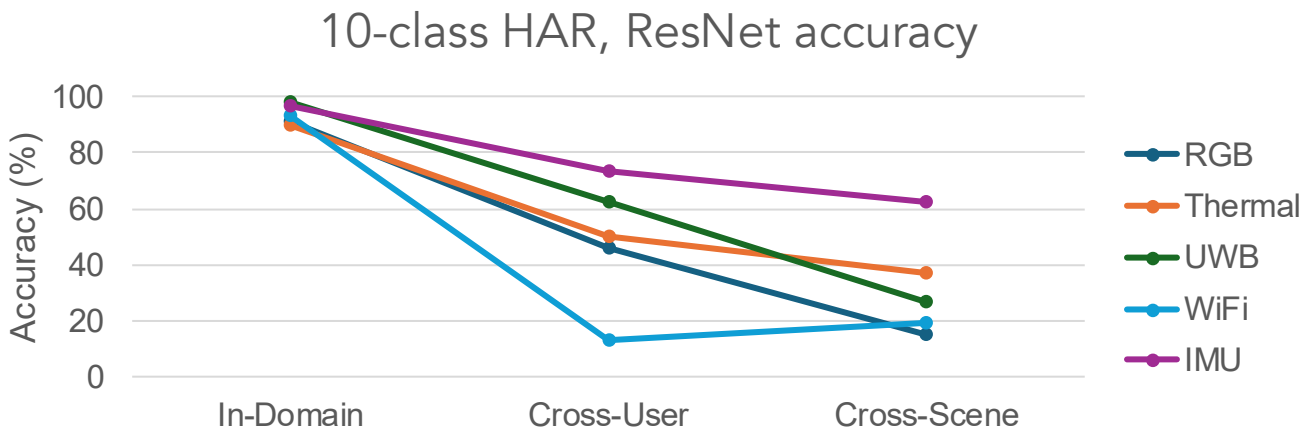
Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

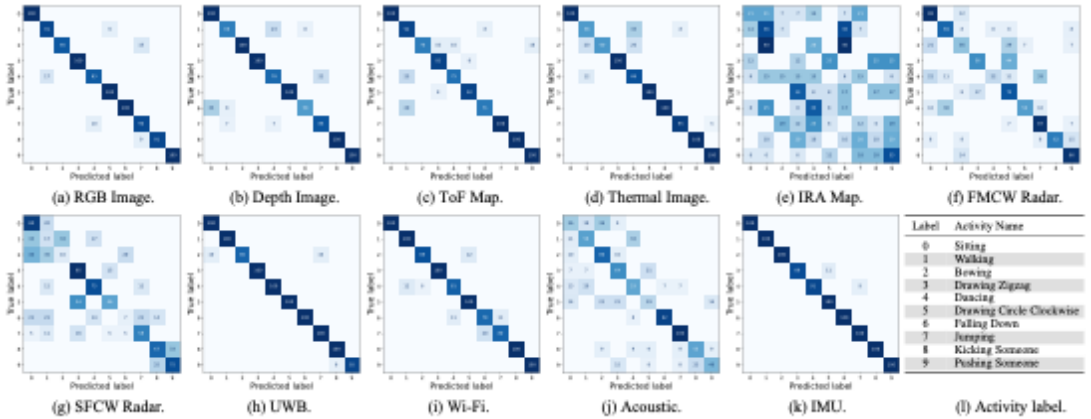
RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

Web Page
https://aiot-lab.github.io/OctoNet/

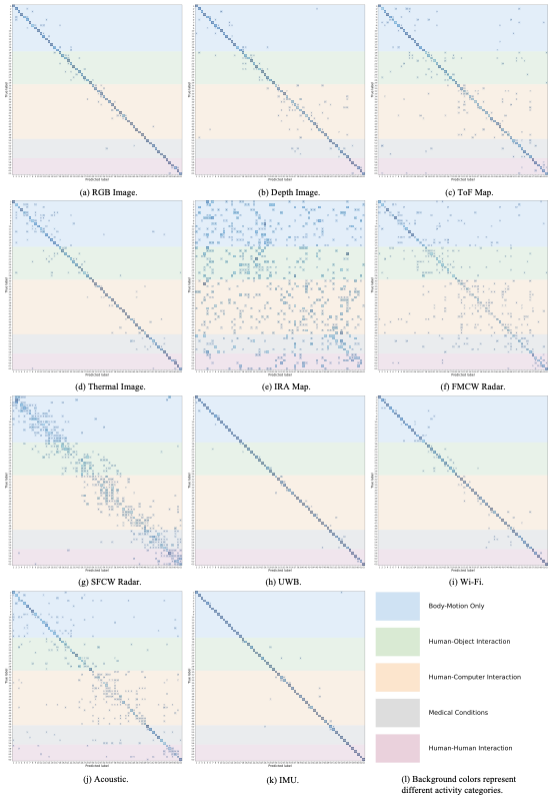
Note-book
https://github.com/aiot-lab/OctoNet_demo

Bech-mark
https://github.com/aiot-lab/OctoNet





10-classs HAR (in-domain)



62-class HAR (in-domain)

What OctoNet delivers

Dataset | Website | Jupyter Notebook | Benchmark

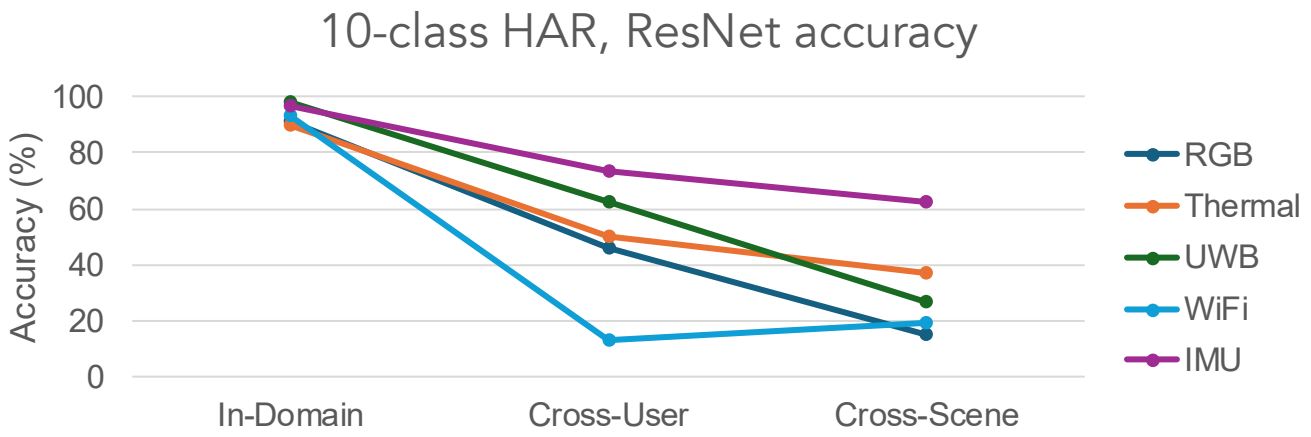
Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

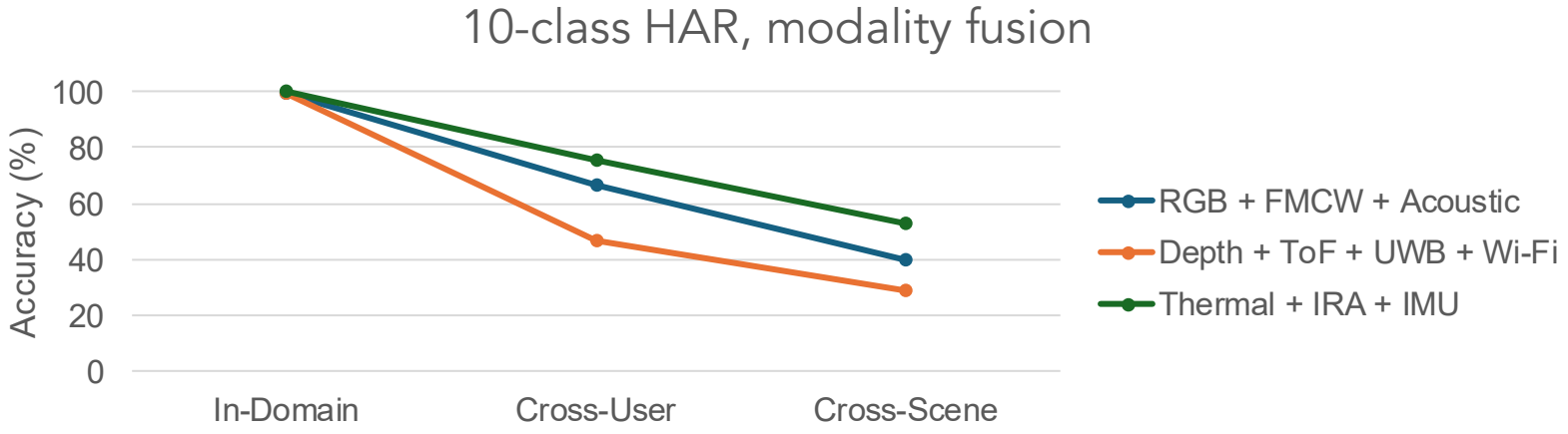
RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

Web Page
https://aiot-lab.github.io/OctoNet/

Note-book
https://github.com/aiot-lab/OctoNet_demo

Bech-mark
https://github.com/aiot-lab/OctoNet





Moving beyond how we fuse modalities, how should we selectively choose which modalities to fuse?

What OctoNet delivers

Dataset Website Jupyter Notebook Benchmark

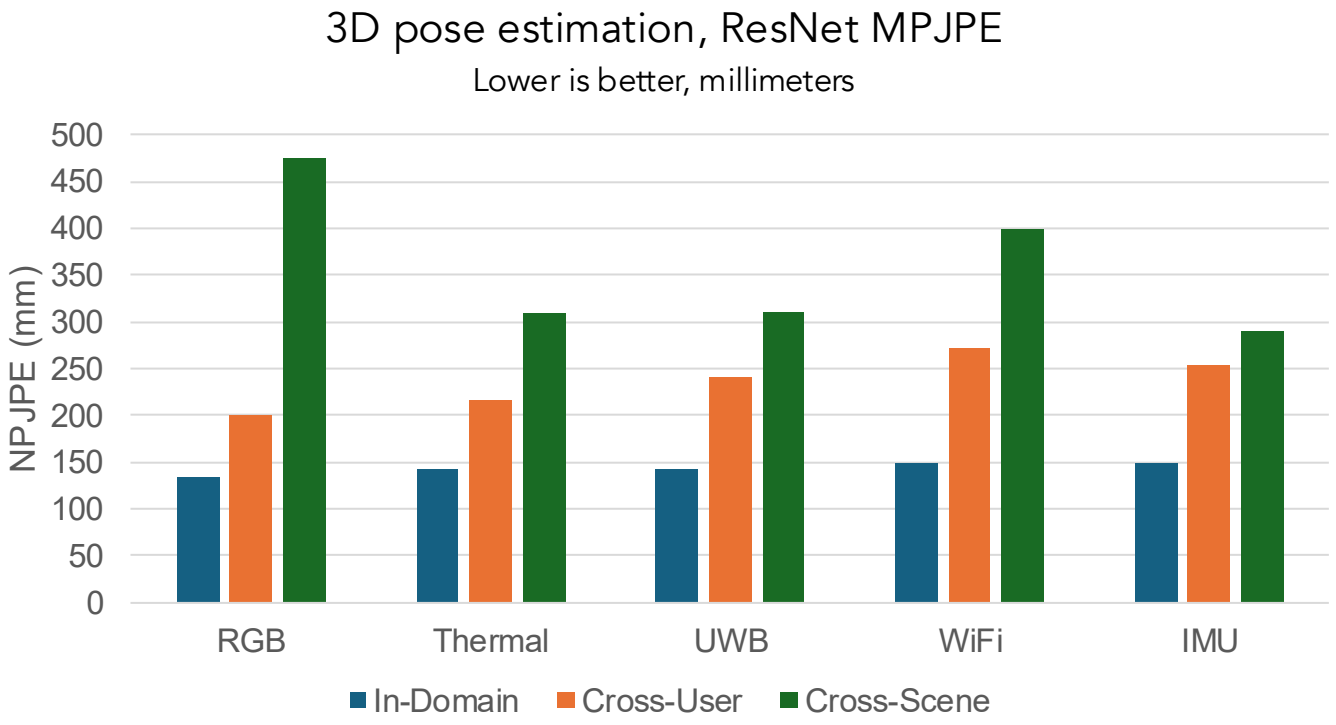
Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

Web Page
https://aiot-lab.github.io/OctoNet/

Note-book
https://github.com/aiot-lab/OctoNet_demo

Bech-mark
https://github.com/aiot-lab/OctoNet





The most precise sensor in familiar conditions is not necessarily the most robust sensor under domain shift.

OctoNet: Future Dataset Infrastructure

Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

Web Page
https://aiot-lab.github.io/OctoNet/

Note-book
https://github.com/aiot-lab/OctoNet_demo

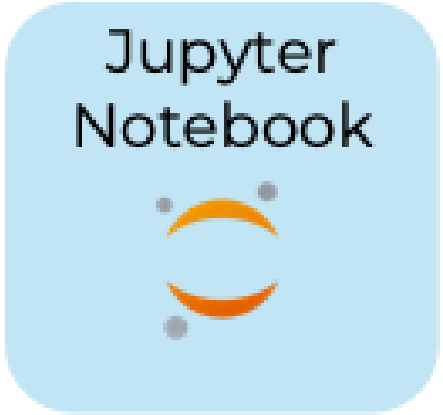
Bech-mark
https://github.com/aiot-lab/OctoNet



- **Streamable Directory Layout:** Flat list structure designed for effortless streaming and selective downloading.
- **Metadata-Rich Naming:** Self-descriptive naming schema incorporating complete file metadata.
- **Privacy & Access Control:** Sensitive modalities (e.g., RGB) are managed via Hugging Face Gated Datasets, requiring approved access requests.



- **Interactive Dataset Portal:** Features dataset specifications, collection videos, and sensor setup visuals.
- **Smart Data Selection Tool:** Interactive web utility to filter data subsets and generate configuration files for seamless streaming downloads.

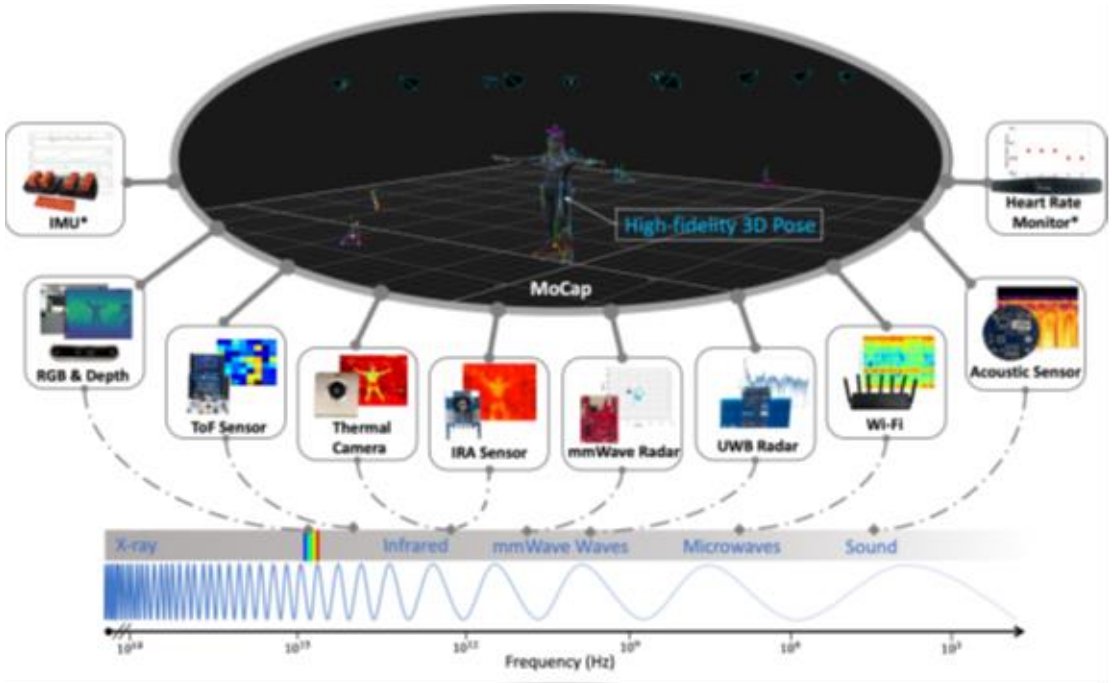


- **Full Multi-Modal Demos:** Executable Jupyter notebooks demonstrating streaming downloads, sample loading, and visualization.
- **Instant Quick-Start:** Out-of-the-box pipeline that abstracts away background streaming, data loading, and preprocessing.



- **Open-Source Benchmark:** Evaluates both single-modality baselines and multi-modal fusion strategies.
- **Rigorous Evaluation:** Standardized evaluation protocols spanning both in-domain testing and cross-domain generalization.

Thank you



Main Dataset
https://huggingface.co/datasets/hku-aiot/OctoNet

RGB Data
https://huggingface.co/datasets/hku-aiot/octonet_rgb

Web Page
https://aiot-lab.github.io/OctoNet/

Note-book
https://github.com/aiot-lab/OctoNet_demo

Bech-mark
https://github.com/aiot-lab/OctoNet

RF Behavior Dataset - Overview

A Multimodal Radio-Frequency Dataset for Activity and Affective Behavior Analysis †

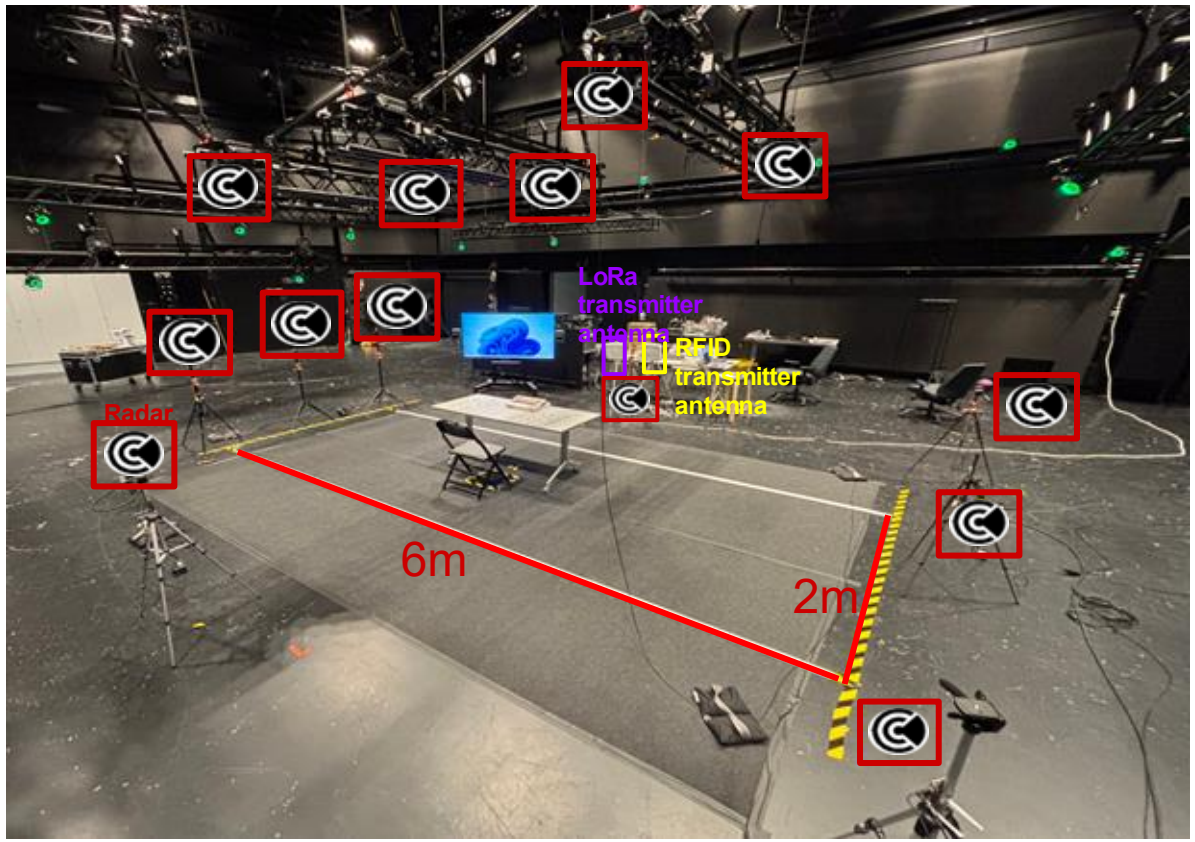
Si Zuo*, Yuqing Song*, Jin Han, Sahar Golipoor, Ying Liu, Xujun Ma, Petter Holme, Stephan Sigg

61
participants

21 gestures
10 activities
6 affective behavior expressions

Laboratory
Living room
Industrial factory

Radar | RFID | LoRa | IMU | Infrared Camera



† Proc. ACM IMWUT, to appear (Vol. 10, No. 4, Dec 2026)
https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Overview

A Multimodal Radio-Frequency Dataset for Activity and Affective Behavior Analysis †

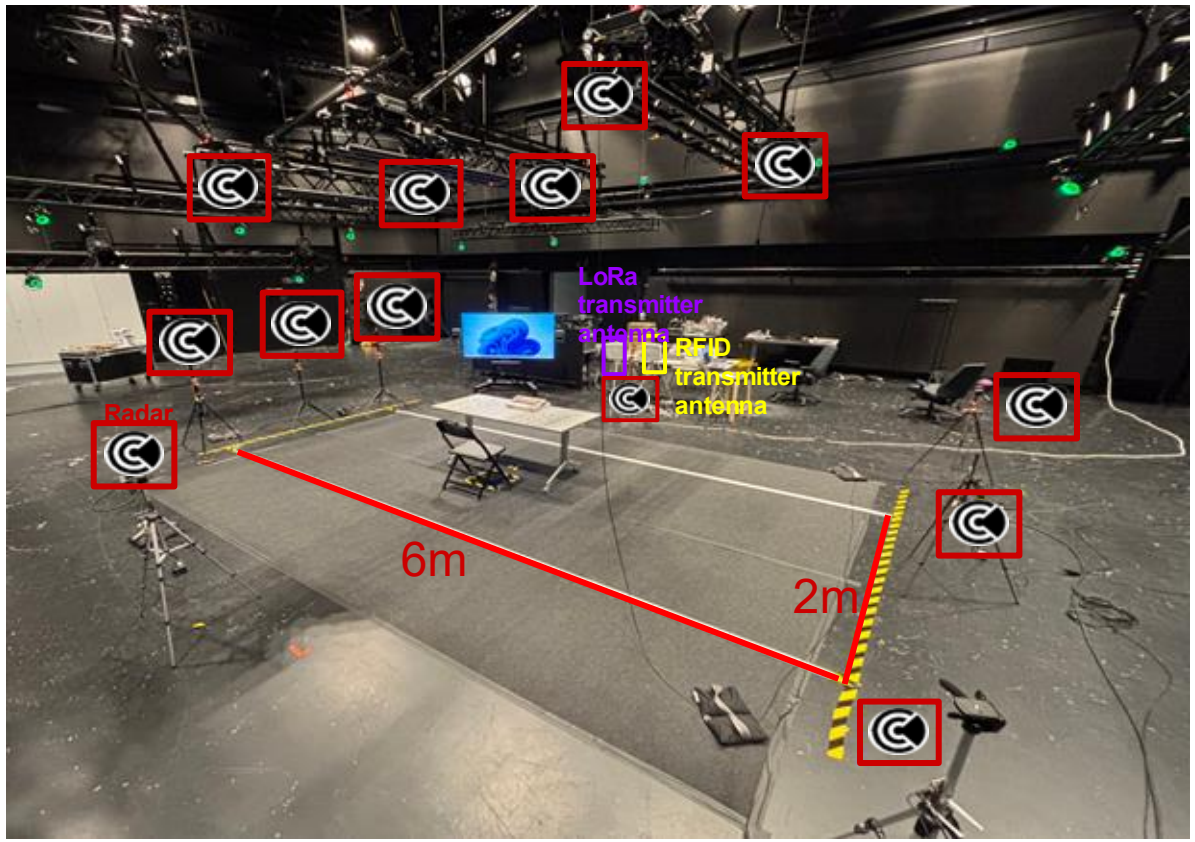
Si Zuo*, Yuqing Song*, Jin Han, Sahar Golipoor, Ying Liu, Xujun Ma, Petter Holme, Stephan Sigg

61 **+7**
participants

21 gestures
10 activities
6 affective behavior expressions

Laboratory
Living room
Industrial factory

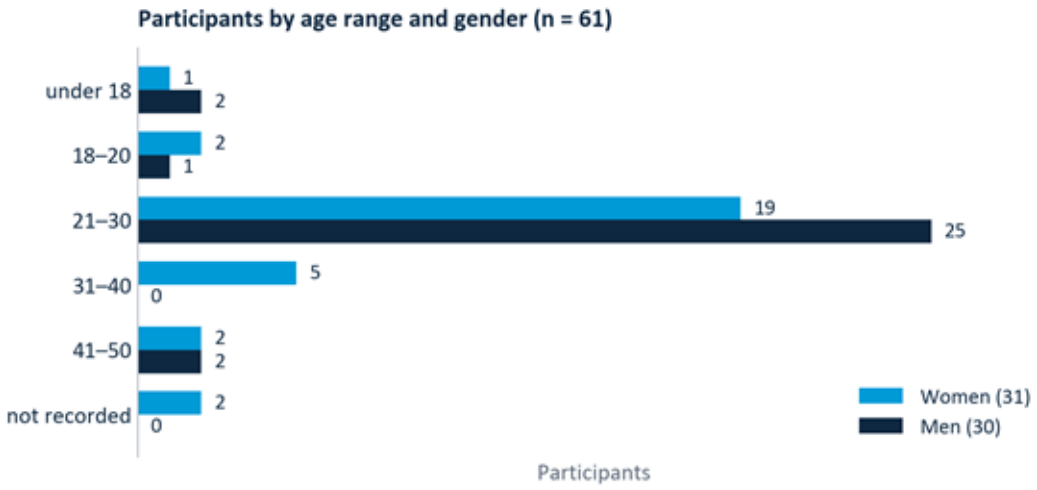
Radar | RFID | LoRa | IMU | Infrared Camera | ECG | EEG



† Proc. ACM IMWUT, to appear (Vol. 10, No. 4, Dec 2026)
https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset

Participants: demographics and anthropometrics



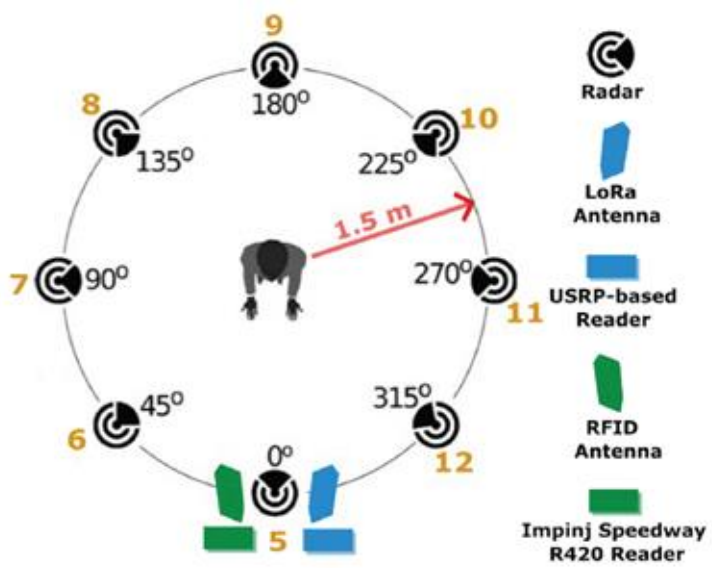
Body measures, measured at the session (examples from meta/participants.csv)

Participant	Shoulder width (cm)	Head-to-shoulder (cm)	Arm length (cm)
U01	41	25	57
U03	44	28.5	58
U07	42	26	53
U10	33.5	25	43
U14	38	25	52
U28	44.5	26.5	55
U45	42.5	23.5	53.5
U48	47	25	56
U50	40.2	25.5	63.5
U57	48	27.2	58.5

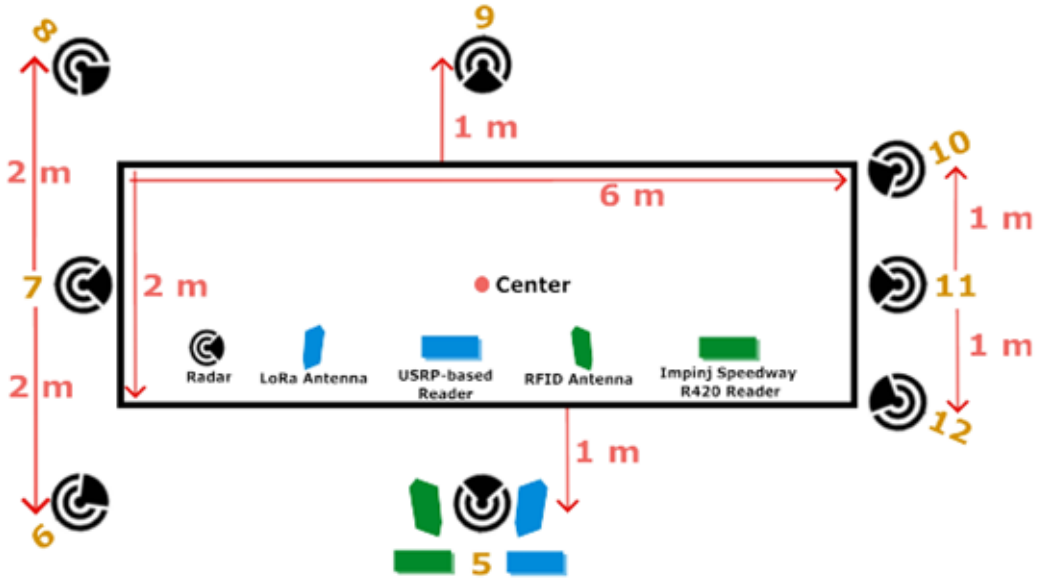
https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Sensor Placement

Gesture campaign



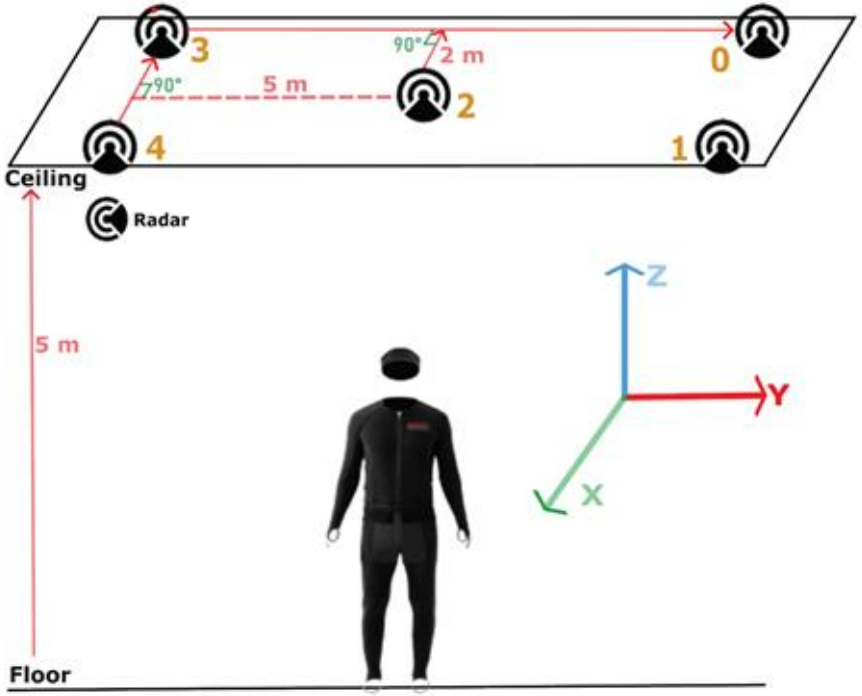
Activity and affective behavior campaigns

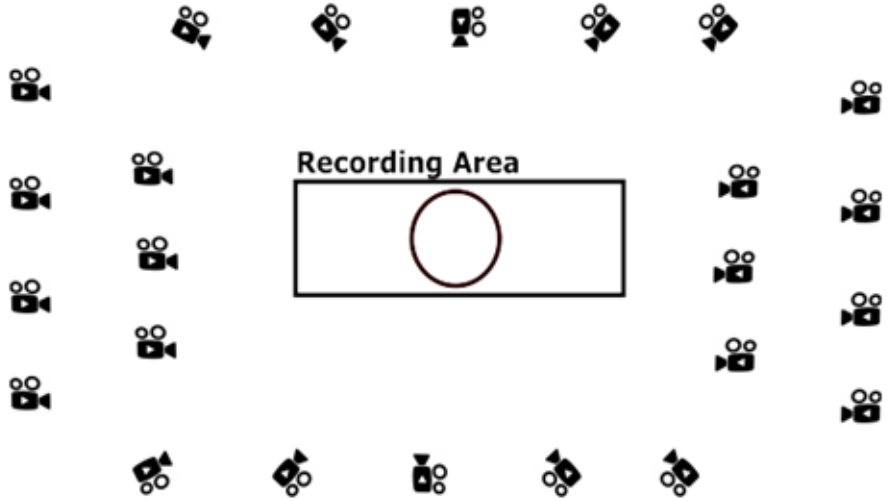


Ground configuration

https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Sensor Placement





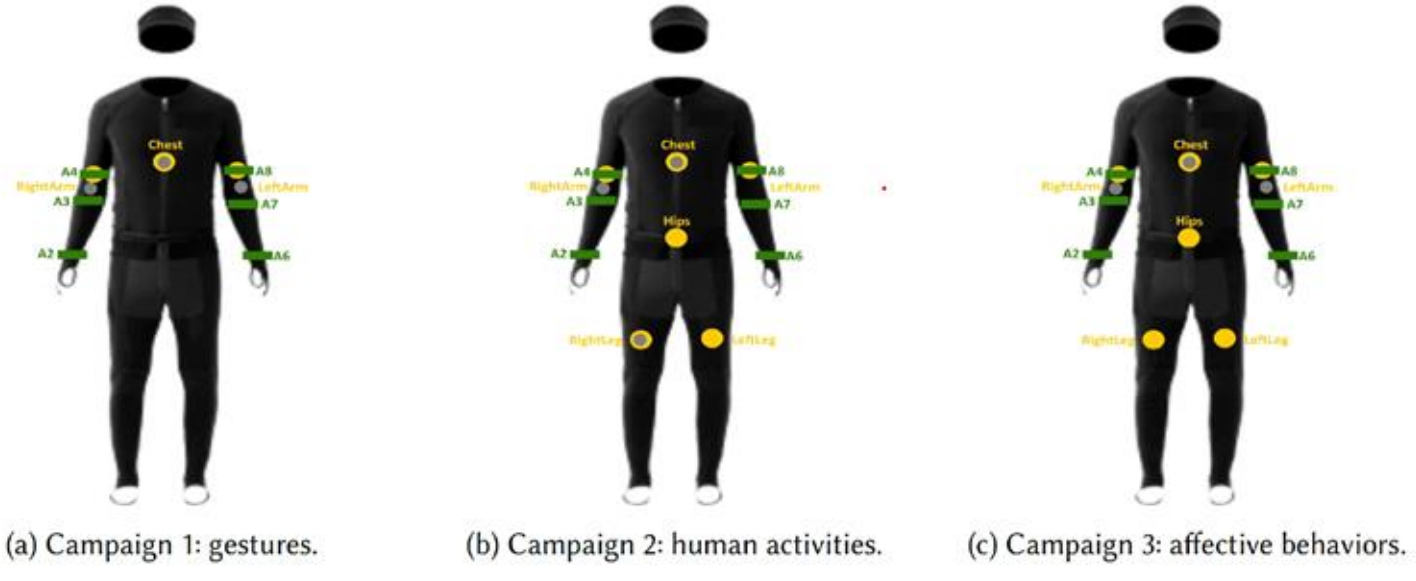
24 synchronized infrared cameras
Reconstruct three dimensional marker trajectories100 Hz sampling

Ceiling and suspended configuration

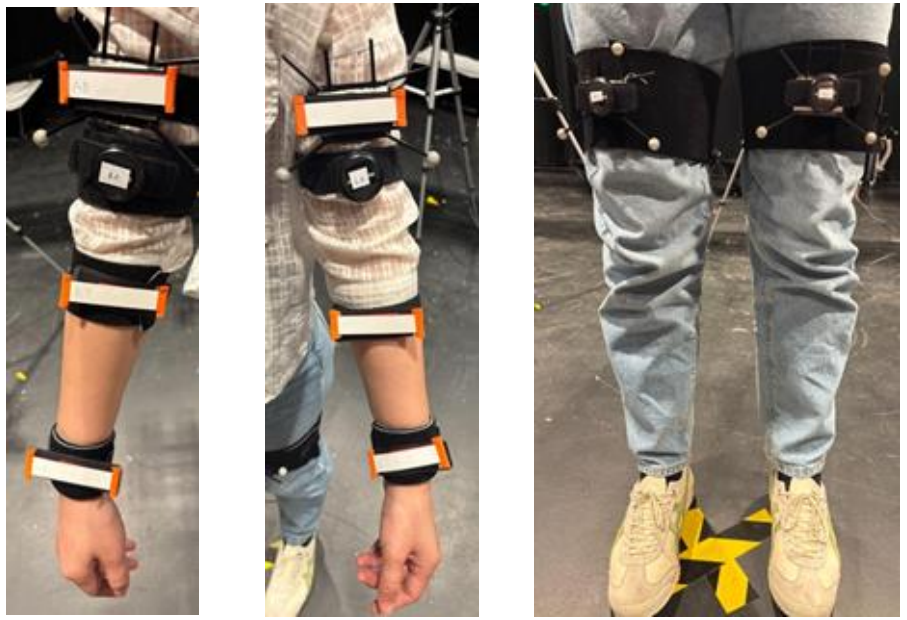
https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Sensor Placement

On-body configuration



(a) Campaign 1: gestures. (b) Campaign 2: human activities. (c) Campaign 3: affective behaviors.



Green: RFID tags, track limb level backscatter at wrists and arms
Yellow: Infrared camera markers, provide groundtruth for motions
Gray: IMU sensors, provide acceleration, angular velocity, and orientation

https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Categories

Gestures: 21 classes; hand and arm motion; fine spatial detail; high orientation sensitivity

lateral-raise, push-down, lift, pull, push, lateral-to-front, swipe-right, swipe-left, throw, arms-swing, two-hand-throw, two-hand-push, two-hand-pull, two-hand-lateral-raise, left-arm-circle, right-arm-circle,

two-hand-outward-circles, two-hand-inward-circles, two-hand-lateral-to-front, circle-clockwise, circle-counter-clockwise

Activities: 10 classes; whole body locomotion and sports; large movement range; strong global translation

walking, running, sitting, lying, ascending stairs, descending stairs

passing a basketball, playing badminton, playing floorball, kicking a football

Affective behavior: 6 expressions; subtle temporal change; labels based on the experimental task and self report

focus, distraction, stress, relaxation, depression, and excitement

https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Affective behavior collection

Part 1: focus, distraction, stress, relaxation (with activity: do a jigsaw puzzle)
Part 2: depression, excitement (role play)

* Each participant involves in affective behavior collection once; some participants are under 18, they only attended Part 1

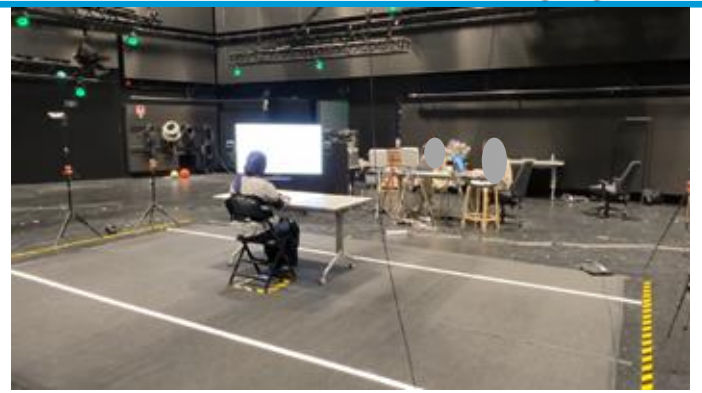


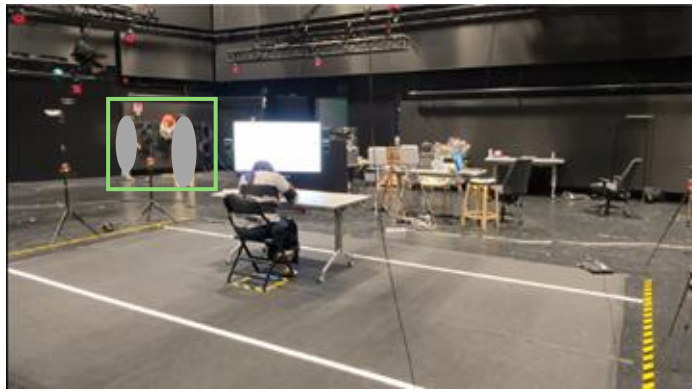
https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Affective behavior collection

Part 1 dataset collection: **Focus vs. Distraction vs. Stress vs. Relaxation**

- Scenario: The participant sits on a chair and **does a jigsaw puzzle**. Pieces are scattered across a 2-meter-wide desk surface to ensure large, sensor-detectable movements when reaching for pieces.
- The collection process is continuous without stopping the participant, and it lasts 8-11 minutes:
  - Focus case (2 min recording)
    - The participant is asked to put jigsaw puzzle together without any requirements.
  - Distraction case (3-5 min recording)
    - Level 1: Random noise and phone ringtone (every 30 sec).
    - Level 2: level 1 + talk interrupt
    - Level 3: level 1 + level 2 + do sports
  - Stress case (3 min)
    - Inform the participant that “you have now only 3 minutes to finish it”.
    - A countdown timer shown on the screen with verbal counting.
    - Oral reminder for the last 10 seconds.
  - Relaxation case (2 min)
    - Inform the participant that now there is no time pressure, build in their own speed.
- Post-Task Assessment:
  - Self-reported focus/distraction/stress/relaxation level (0–5 scale).







https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Affective behavior collection

Part 2 dataset collection: **Depression vs. excitement (role play)**

- Scenario: The participant watches emotion-inducing videos, and **shares stories** while taking the emotion into their talks and body gestures.
  - Responds to neutral prompts by taking the described feeling into their talks and gestures.
    - Video about depression:
    - https://www.youtube.com/watch?v=EJ_S5Rjt_iI
    - Video about excitement:
    - https://www.youtube.com/watch?v=ViSVOWv3Pp8
- Questions:
  - Describe an experience you've had with
    - an animal (either a pet or a wild animal).
    - Sports
    - *if there is no relevant stories, the participant is free to share any story related to the emotion
  - *Note:
    - The content of the answer from participants will not be for analysis.
    - The participant is encouraged to talk in their preferred language.
- Post-Task Assessment: self-reported depression and excitement level (0–5 scale).





https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Affective behavior collection

Campaign 3: self-assessment after each affective state (examples)

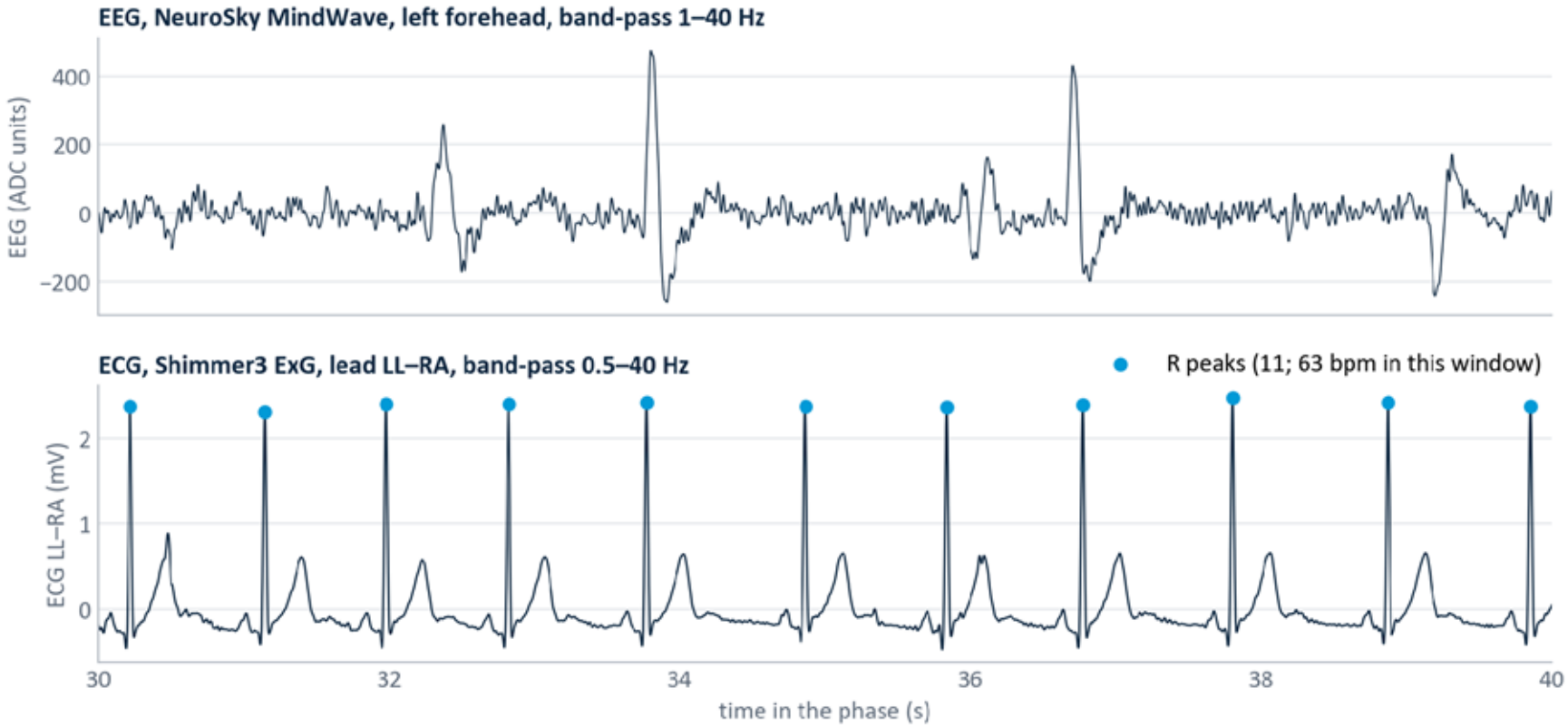
Participant	Focus	Distraction	Stress	Relaxation	Depression	Excitement
U03	4	3	3	4	3	3
U07	5	3	3	3	5	3
U08	3	2	1	3	4	5
U11	5	2	2	3	2	5
U14	4	2	3	5	5	4
U22	4	4	4	3	4	5
U48	3	3	2	3	5	4
U50	4	5	5	3	3	1
U55	4	2	1	5	5	1
U58	4	3	4	5	5	4

Scale: 0 = not in the mood, 1 = slightly, 2 = somewhat, 3 = moderately, 4 = mostly, 5 = quite in the mood.

https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Affective behavior collection

Campaign 3: **a physiological reference subset with additional ECG and EEG sensors**





EEG headset



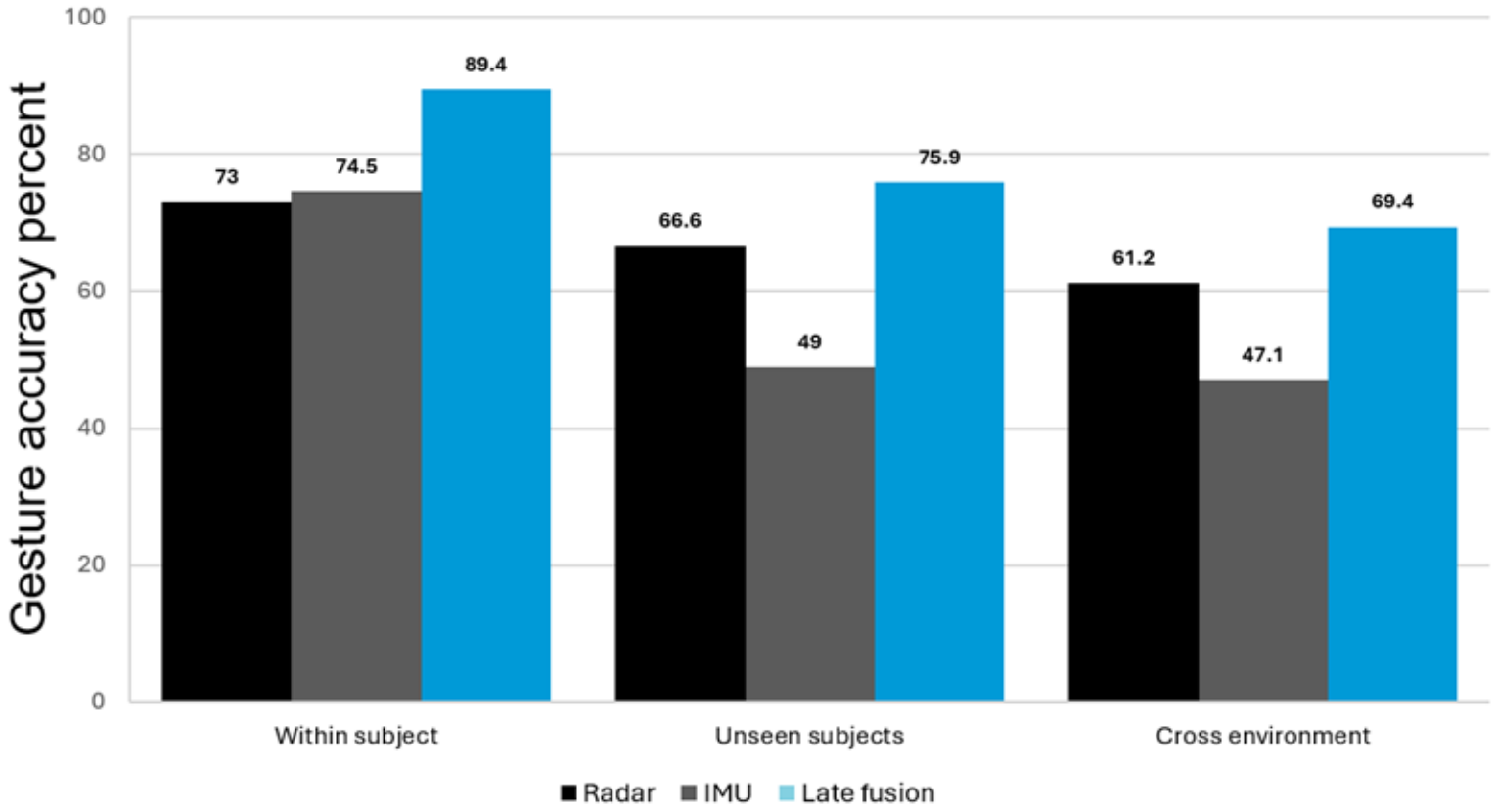
ECG Unit

https://huggingface.co/datasets/Si-Z/RF-Behavior

EEG headset: https://store.neurosky.com/pages/mindwave
ECG Unit: https://www.shimmersensing.com/product/shimmer3-ecg-unit-2/

RF Behavior Dataset - Evaluation

Subject shift and environment shift



- Fusion helps under subject and environment shift.
- Radar degrades less than IMU.

https://huggingface.co/datasets/Si-Z/RF-Behavior

Cross-environment adaption

Cross-environment activity recognition accuracy (%) for radar-only baselines and radar and IMU fusion. N denotes the number of Scenario 2 living-room subjects added for adaptation.

Protocol	Model / Fusion	Scenario 2: Living Room				Scenario 3: Factory			
		$N=0$	$N=2$	$N=4$	$N=6$	$N=0$	$N=2$	$N=4$	$N=6$
Radar Only	PointNet++	47.4	53.0	60.0	62.0	44.7	37.0	39.0	37.0
	Tesla	29.8	49.0	54.0	50.0	34.2	24.0	42.0	37.0
radar and IMU, Fine-tune	PointNet++ / Cross	–	42.3	71.7	61.0	–	90.0	85.0	75.0
	PointNet++ / Feature	–	44.9	44.7	64.9	–	85.0	80.0	80.0
	PointNet++ / Late	–	100.0	100.0	98.7	–	90.0	90.0	85.0
	Tesla / Cross	–	40.7	59.9	67.5	–	90.0	80.0	95.0
	Tesla / Feature	–	29.4	59.9	63.6	–	90.0	90.0	80.0
	Tesla / Late	–	100.0	100.0	100.0	–	90.0	90.0	90.0
radar and IMU, Retrain	PointNet++ / Cross	–	70.1	79.0	76.6	–	84.2	86.8	86.8
	PointNet++ / Feature	–	54.1	61.2	71.4	–	68.4	86.8	86.8
	PointNet++ / Late	–	100.0	100.0	100.0	–	89.5	89.5	89.5
	Tesla / Cross	–	61.9	51.3	62.3	–	76.3	31.6	81.6
	Tesla / Feature	–	47.9	70.4	64.9	–	86.8	86.8	84.2
	Tesla / Late	–	100.0	100.0	100.0	–	89.5	89.5	89.5

Key finding: Late fusion is consistent across both environments. With two adaptation subjects, accuracy reaches 89.5% to 100%.

https://huggingface.co/datasets/Si-Z/RF-Behavior

RF Behavior Dataset - Evaluation

Affective behavior evaluation uses personalized adaptation

Affective behavior recognition: Few-Shot Adaptation across Models and Input options for IMU data.



Few shot subject adaptation

- One to ten labeled reference samples per behavior and subject.
- Largest improvement usually appears between one and two samples.

Recognition accuracy with CNN:

Radar: 75.8%
IMU: 75.2%
Radar + IMU: 80.6%

https://huggingface.co/datasets/Si-Z/RF-Behavior





Thank you.

OctoSense: A Unified Ecosystem for Open-Source Wireless Sensing

Weiying Hou, Xie Zhang, Chenshu Wu

From datasets to experiments

How can we use these datasets in our experiments?

How can we apply existing processing methods and models?

```python
import octosense as octo

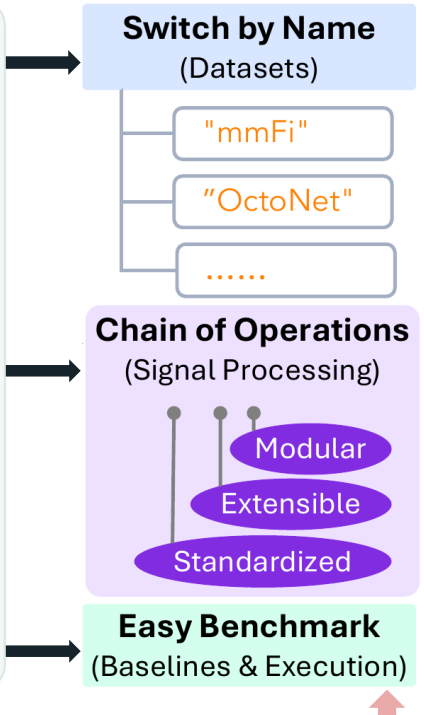
# Load dataset
dataset = octo.dataset.load("widar3", modality="CSI")

# Define declarative signal processing pipeline
transforms = octo.transform.compose(
    octo.transform.csi.pilot_subc_interp(method="linear"),
    octo.transform.axis_normalize(axis="subc"),
    octo.transform.magnitude(),
)

# Build dataloaders
train_loader, test_loader = octo.loader.build(
    dataset, transform=transforms, dataset_split=(8,2)
)

# Init model and run benchmark
model = octo.models.stfnet(pretrained=False)
octo.benchmark.run(model, train_loader, test_loader)
```

Workflow overview



Saving Experiment Parameters in a Specification File

stfnet_widar3.spec

Reproduce w/ Command

```
(base) ➜ ~ octosense exec
--spec stfnet_widar3.spec
```

Motivation: Why RF reuse is challenging

1 **Device formats**

File formats, field names and axis layouts

Locate the signal and interpret its stored dimensions.

2 **RF semantics**

Time, subcarriers, sampling rate and antennas

Identify the physical information required by the calculation.

3 **Input assumptions**

Processing requirements and model layouts

Adapt the data to the input required by the method.

Motivation: Shared infrastructure for reuse

Hugging Face: Shared infrastructure

```
from datasets import load_dataset
from transformers import AutoModel

data = load_dataset(
   "stanfordnlp/imdb")
model_id = "google-bert/bert-base-uncased"
model = AutoModel.from_pretrained(model_id)
```

Independent dataset and model examples

Standard transform libraries

Compose preprocessing operations in execution order.

Widar3: Dataset-specific logic

```
1 data_1 = scio.loadmat(file_path)[
     "velocity_spectrum_ro"]
2 label_1 = int(data_file_name.split("-")[1])
  location = int(data_file_name.split("-")[2])
  orientation = int(
     data_file_name.split("-")[3])
3 if label_1 not in motion_sel:
     continue
  data_normed_1 = normalize_data(data_1)
4 T_MAX = max(T_MAX, data_1.shape[2])
```

1. **BVP field**
2. **Labels and conditions**
3. **Sample preparation**
4. **Time-axis assumption**

Track the same physical time axis across each operation

```
data_1_max = np.concatenate(
    (data_1.max(axis=0), data_1.max(axis=1)),
    axis=0).max(axis=0)
data_1_max_rep = np.tile(data_1_max,
    (data_1.shape[0], data_1.shape[1], 1))

t = np.array(data[i]).shape[2]
data_pad.append(np.pad(data[i],
    ((0, 0), (0, 0), (T_MAX - t, 0)),
    "constant", constant_values=0).tolist())

data = np.swapaxes(
    np.swapaxes(data, 1, 3), 2, 3)
data = np.expand_dims(data, axis=-1)
```

Sample: [v_x, v_y, time] → Model batch: [batch, time, v_x, v_y, channel]

Official normalize_data, zero_padding and load_data (selected statements)

1. Normalize

Reduce velocity axes. Retain one value per time step.

2. Pad time

shape[2] selects time. The third padding pair extends it.

3. Match model layout

Time follows batch. A channel axis is added.

MOTIVATION / Dataset, preprocessing and model must agree

Models rely on consistent axis meanings

```
inputs = Input(shape=(T_MAX, 20, 20, 1))
conv = Conv2D(16, (5, 5), activation="relu",
              data_format="channels_last")
x = TimeDistributed(conv)(inputs)
x = TimeDistributed(MaxPooling2D((2, 2)))(x)
x = TimeDistributed(Flatten())(x)
x = TimeDistributed(Dense(64, activation="relu"))(x)
x = TimeDistributed(Dropout(0.5))(x)
x = TimeDistributed(Dense(64, activation="relu"))(x)
x = GRU(128, return_sequences=False)(x)
x = Dropout(0.5)(x)
outputs = Dense(6, activation="softmax")(x)
model = Model(inputs=inputs, outputs=outputs)
```

Official assemble_model (adapted); both velocity axes have 20 bins

Input layout

Each sample: [time, v_x, v_y, channel]

Per-frame CNN

TimeDistributed keeps time. Conv2D uses the two velocity axes.

GRU layout

[batch, time, features]

Motivation: What an experiment must preserve

1	**Axis meaning**	Preserve the physical meaning and direction of each axis.
2	**Processing choices**	Record the selected spectrum and preprocessing parameters.
3	**Experimental protocol**	Keep sample selection, labels and train/test splits consistent.

GitHub survey: Evidence from community reuse



538 papers

Keyword search and reference expansion from 28 reviews

89 unique GitHub repositories

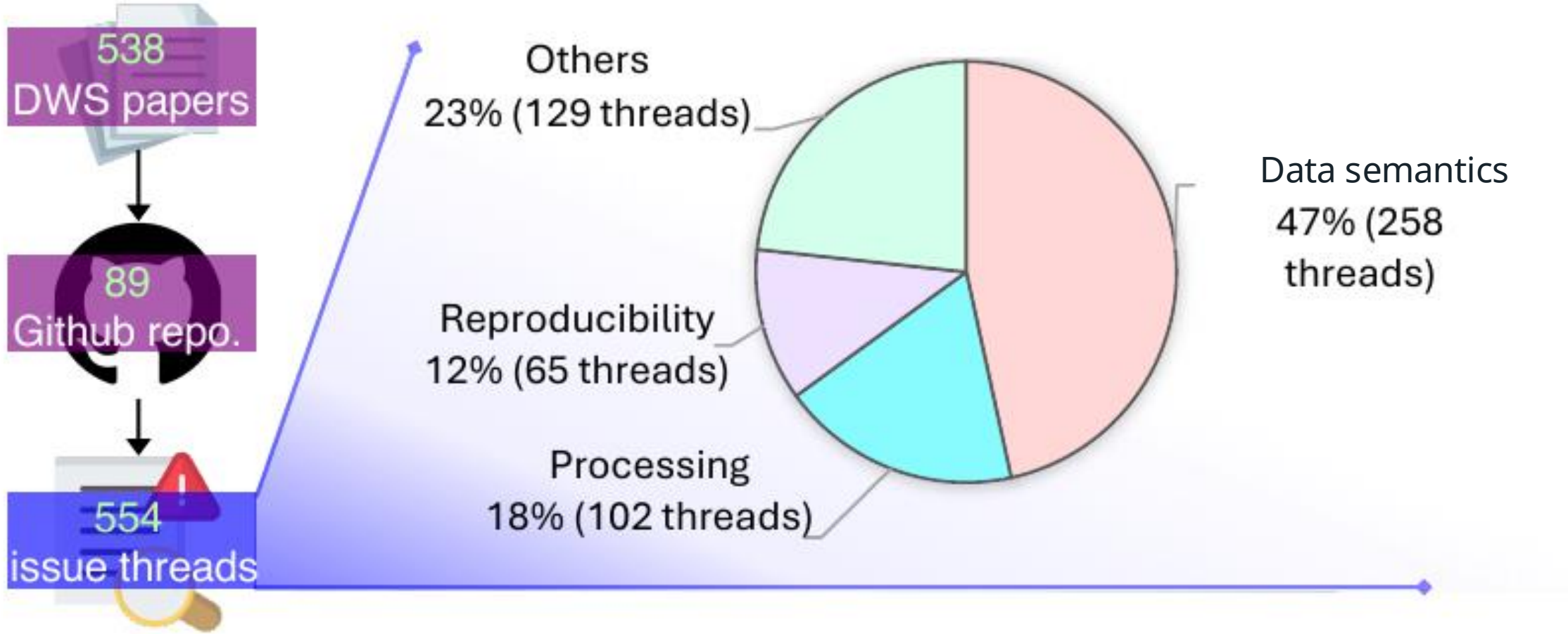
61 repositories contain issue discussions

554 issue threads

LLM candidate extraction followed by manual verification

GitHub survey: Three recurring obstacles

MOTIVATION / Three design priorities from the reviewed discussions



Data semantics → Shared data

Processing → Reusable operators

Reproducibility → Recorded specs

OctoSense

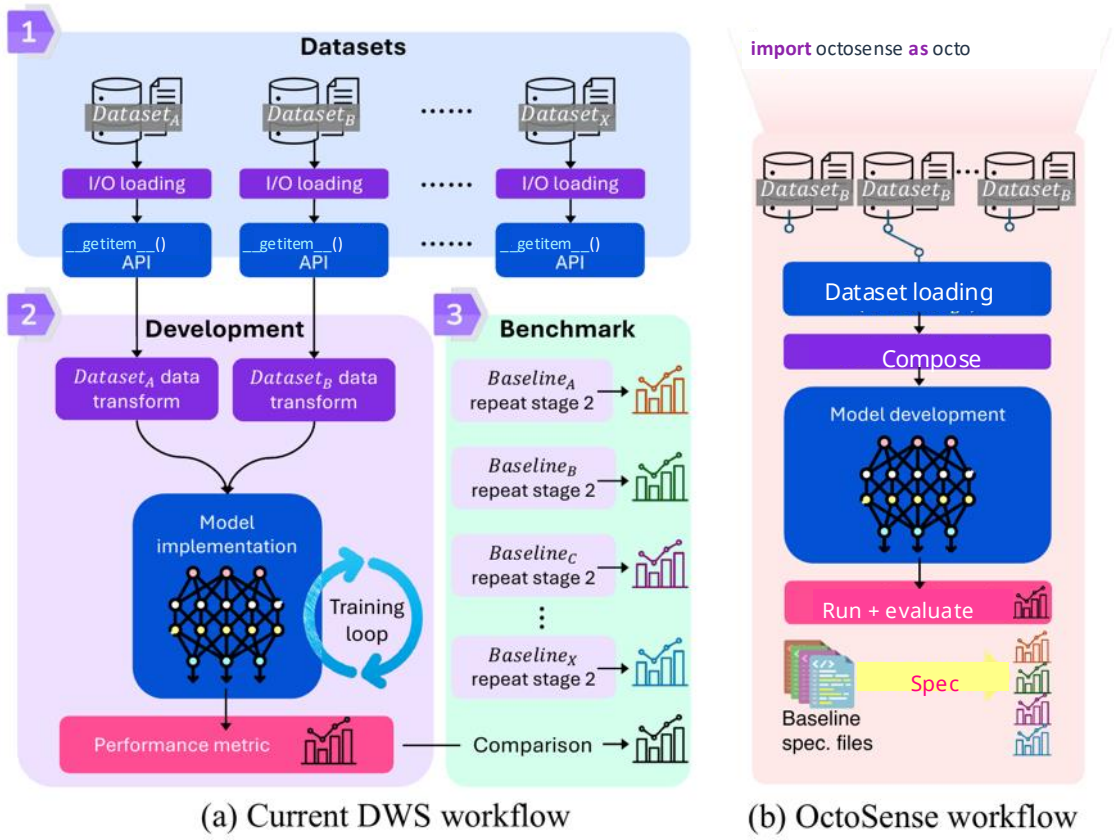
A unified ecosystem for reusable wireless sensing

1. Shared data access — Load RF datasets through a common interface

2. Reusable components — Compose RF processing and connect existing models

3. Repeatable experiments — Run a pipeline, save its specification, then restore it

Design: Three connected decisions



(a) Current DWS workflow

(b) OctoSense workflow

1. Explicit data meaning

Readers attach named axes
and acquisition metadata.

2. Processing and model contracts

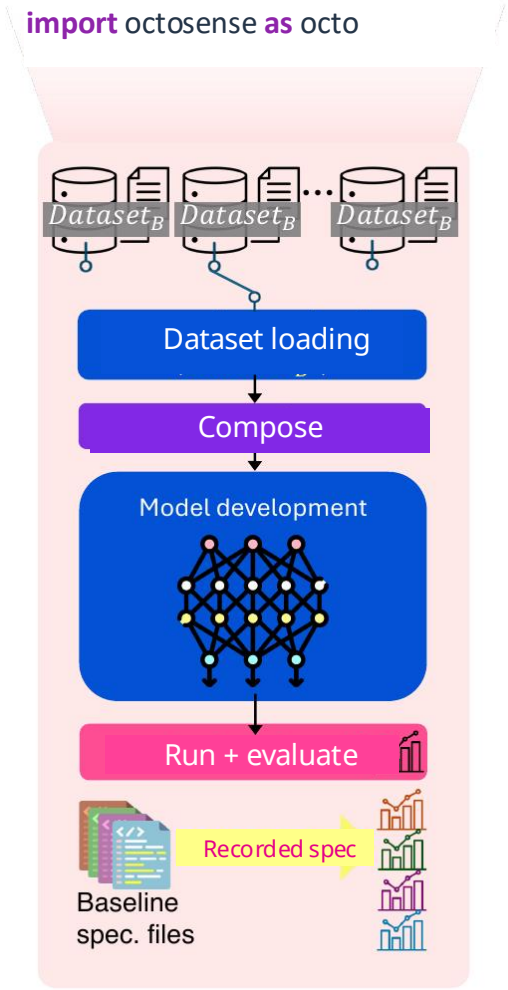
Operators update the representation.
Adapters prepare the declared input.

3. Executable, recorded experiments

Pipelines run the workflow.
Specs preserve its configuration.

OctoSense architecture

DESIGN / Shared components across the workflow



1 Readers

Decode device formats and map RF fields.

2 Dataset views

Select samples and keep task labels aligned.

3 Operators

Reuse signal-processing steps with explicit contracts.

4 Model adapters

Connect data semantics to model input dimensions.

Pipelines execute the composition; specs record it.

OctoSense in use: A reusable experiment

DESIGN AND USE / Loading, composition and reproducibility

```
import octosense as octo

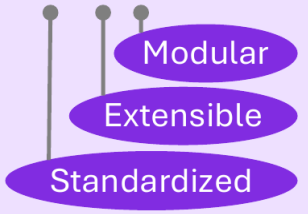
1  view = octo.datasets.load(
       dataset_id="widar3", modalities=["wifi"],
       variant="g6", split_scheme="official",
       task_binding="gesture")

   # spec declares processing, model and runtime
2  pipe = octo.pipelines.load(spec=spec, dataset=view)
3  pipe.auto_align({"time": "time", "feature": "subc*tx*rx"})
4  run = pipe.train()
```

Switch by Name
(Datasets)

- "mmFi"
- "OctoNet"
- .....

Chain of Operations
(Signal Processing)



Easy Benchmark
(Baselines & Execution)

Configured pipeline
Component choices + parameters

→

Saved spec
widar3.yaml

→

Restore configuration
load_yaml(spec_path)

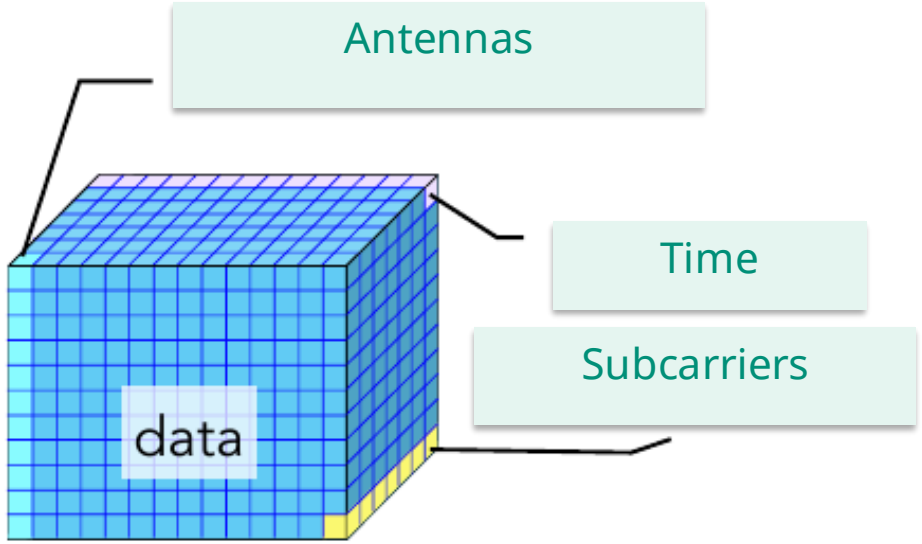
The configuration is expanded in the experiment example.

Data: Signal I/O and metadata management

Signal I/O
Read and represent each recording

Metadata management
Organize, select and split samples

RadioTensor: a common representation for each recording



RF dimensions are identified by physical meaning.

Named axes

time · subc · tx · rx

Acquisition information

Sampling rate, center frequency and antenna configuration

Data: Readers map device formats

Signal I/O
Read and represent each recording

Metadata management
Organize, select and split samples

Map device-specific fields to a shared RF description

Device file

Device-specific format and fields

→

Reader

Decode the file
Assign axis names
Record metadata

→

RadioTensor

Signal values
Named RF axes
Acquisition context

→

Processing

Select time by name
Read sampling rate

Data example: Reading a device capture

Signal I/O
Read and represent each recording

Metadata management
Organize, select and split samples

```python
from pathlib import Path
import octosense as octo

reader = octo.io.readers.load("wifi/iwl5300", channel=36)

sample = reader.read(Path("sample.dat"))

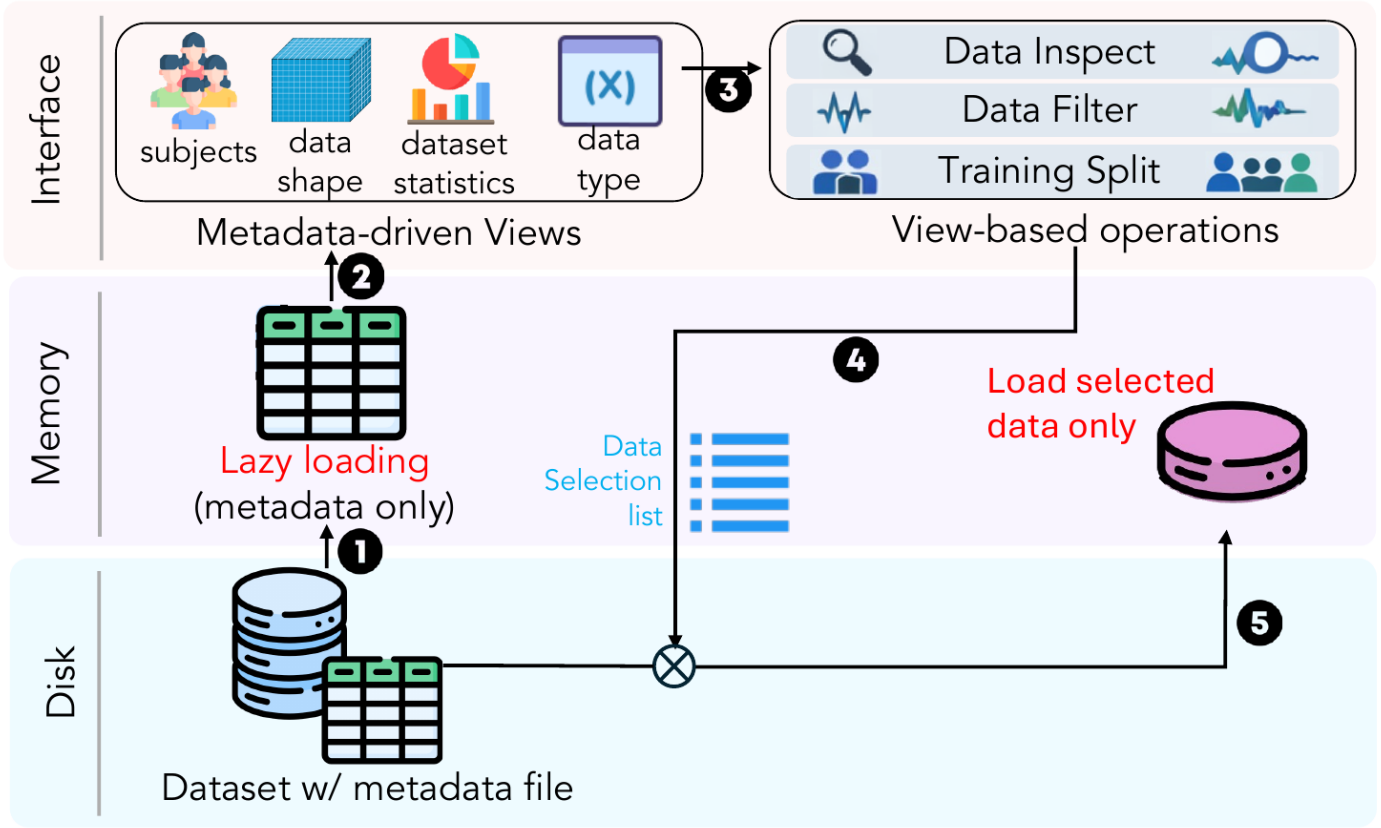
print(sample.axis_schema.axes)  # time, subc, tx, rx
print(sample.metadata.reader_id)  # wifi/iwl5300
print(sample.describe())
```

1. **Choose reader**
   Intel 5300
2. **Read recording**
   RadioTensor
3. **Inspect**
   Named axes and metadata

Data: Metadata management

Signal I/O
Read and represent each recording

Metadata management
Organize, select and split samples



1–2 Read metadata

Subjects, rooms, labels and split assignments

3–4 Select through views

Inspect, filter and split by sample metadata

5 Load selected signals

Load each selected signal with its aligned label

Data example: Select samples, then load

Signal I/O
Read and represent each recording

Metadata management
Organize, select and split samples

```
view = octo.datasets.load(
    "widar3", modalities=["wifi"], variant="g6",
    split_scheme="official", task_binding="gesture",
    path="/data/widar3",
)

test = view.get_split("test")
room1, _ = test.partition(room=1)
print(room1.metadata_rows()[0])  # metadata only

sample, label = room1[0]  # RF signal is loaded here
```

1 Bind the dataset

Variant, split and task

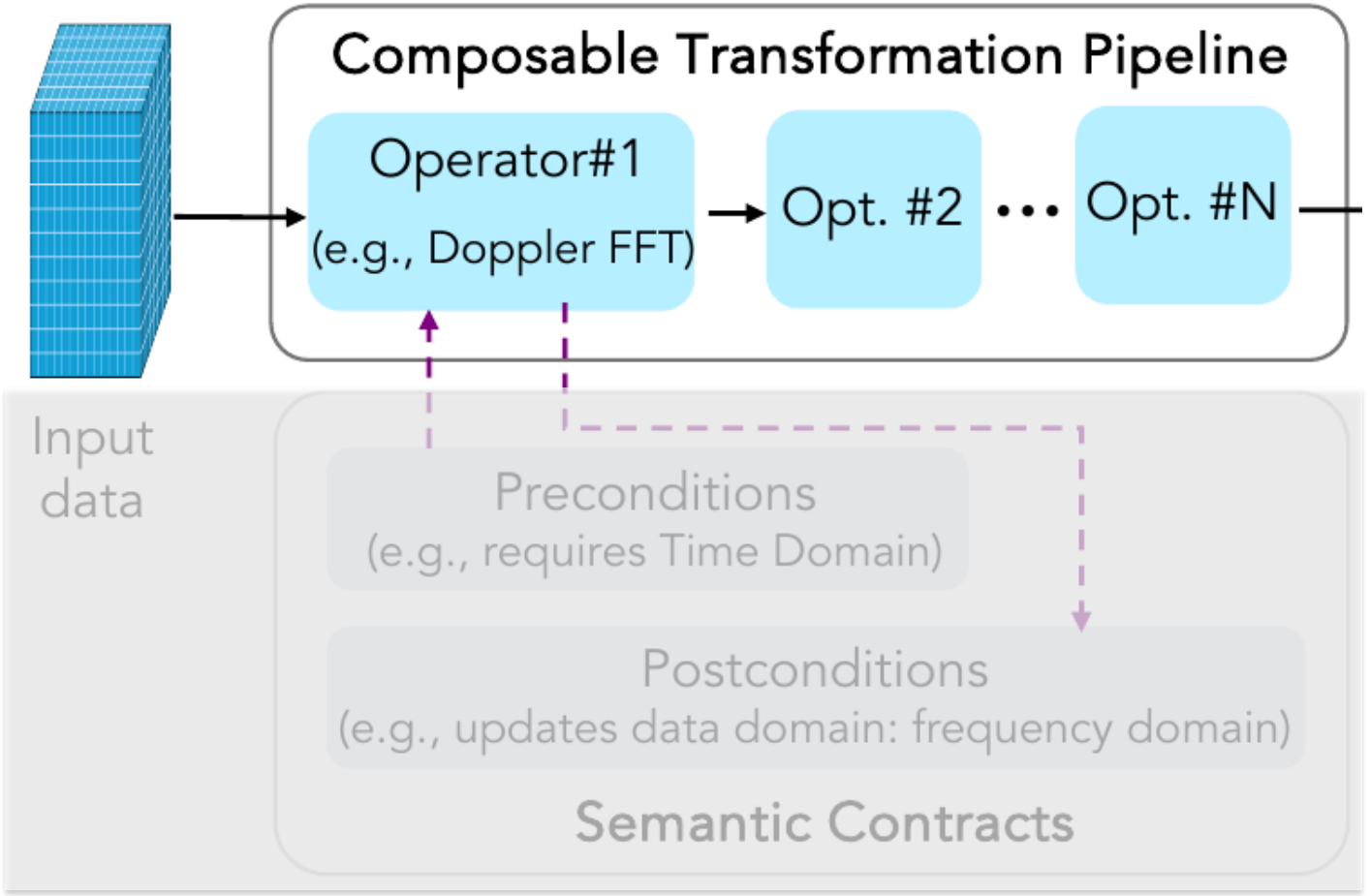
2 Select and inspect

Test split; room one
Inspect metadata

3 Load one sample

Signal + aligned label

Processing: Composable signal operators



Common operator interface

A specified signal field
and its associated metadata

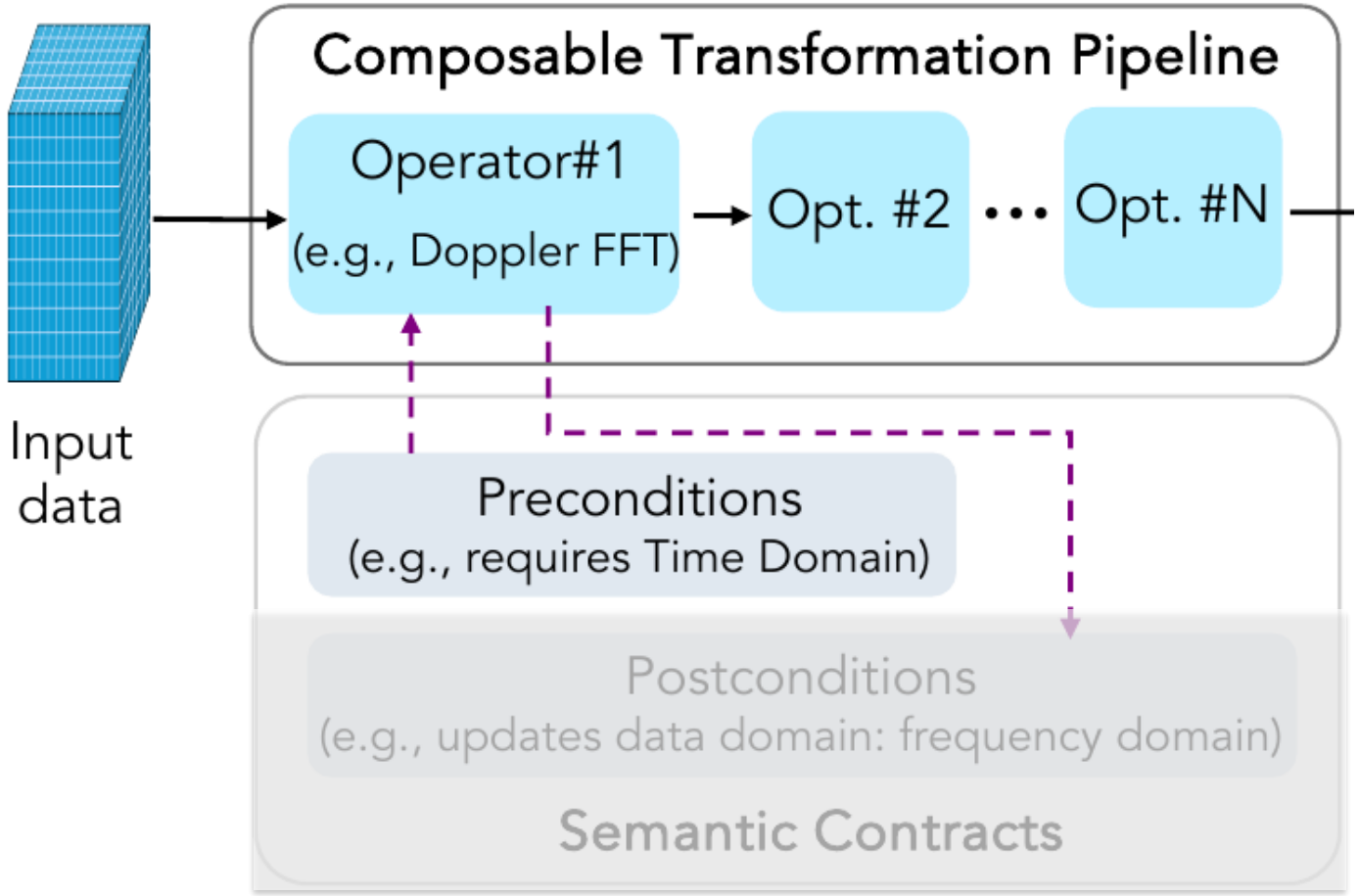
Compose

Run the operators
in the declared order

Semantic contracts

Link output updates
to the next input requirements

Processing: Pre-conditions



Input state

Named axes and metadata
describe the current representation

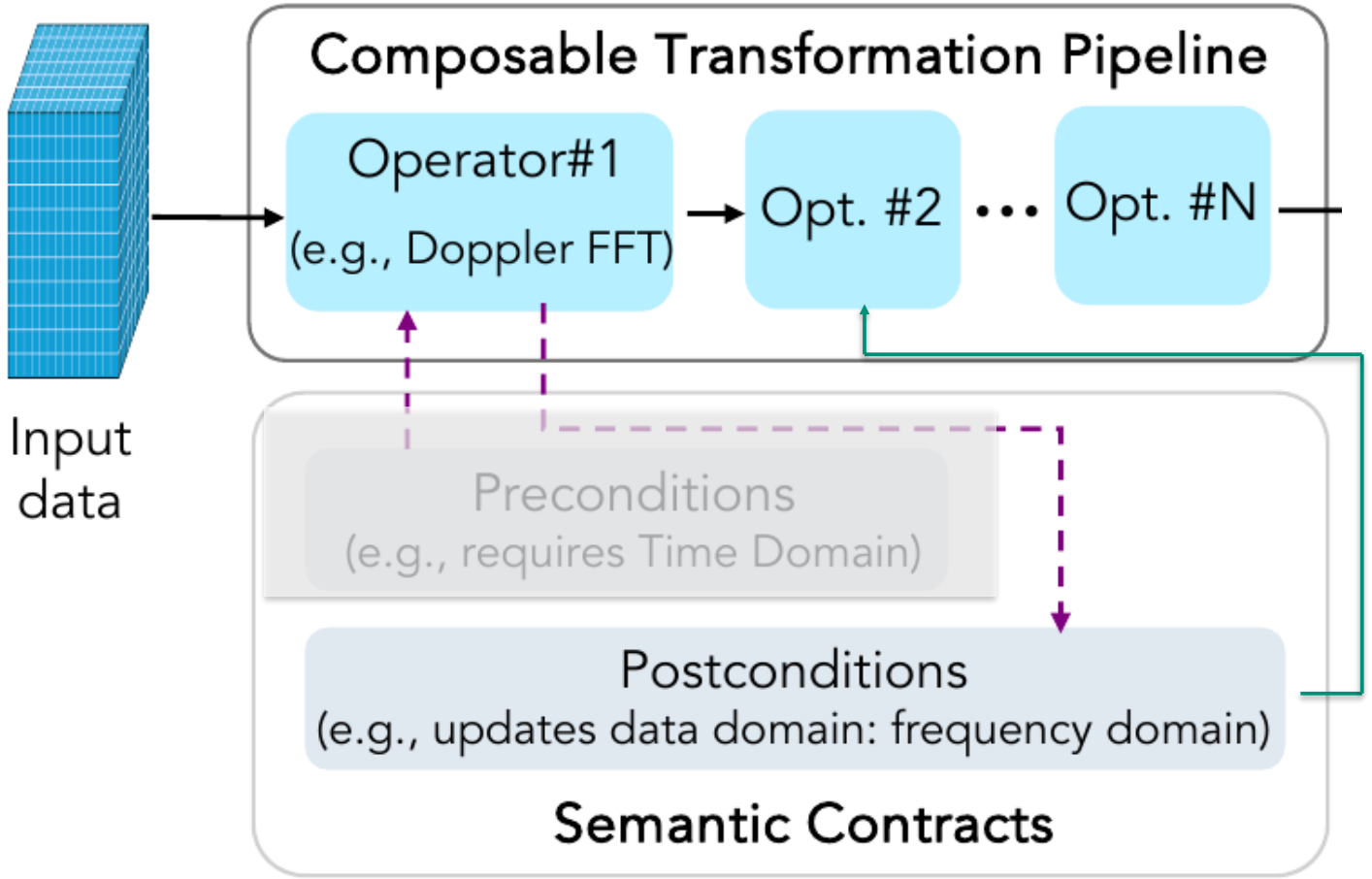
requires

Doppler FFT:
time-domain input

Compatibility check

Match the input state
to the declared requirements

Processing: Post-conditions



Execute the operator

Transform the signal values

updates

Record the new frequency-domain representation

Next operator

Receive the updated state and check its requirements

Processing example: STFT contracts


```
from octosense.transforms.ops.spectral import STFT
op = STFT(axis_name="time", n_fft=128,
          hop_length=64, onesided=False)
```

Pre-condition

```
print(op.requires)
# Requires axis: time
# Requires metadata: sample_rate
```

INPUT REQUIREMENTS

time + sampling-rate metadata

Post-condition

```
print(op.updates)
# Removes axis: time
# Adds axes: freq, frame
```

UPDATED AXES

time → freq + frame

STFT followed by magnitude extraction

```python
from octosense.transforms.ops.complex import Magnitude

preprocess = octo.transforms.compose([
    STFT(axis_name="time", n_fft=128,
         hop_length=64, onesided=False),
    Magnitude(),
])

doppler = preprocess(sample)
print(doppler.axis_schema.axes)
```

1 Compose

List operations in execution order

2 Name the axis

Select the physical time axis

3 Reuse

Processing sequence with updated RF axes

Input: time, subc, tx, rx ⟶ **Output: subc, tx, rx, freq, frame**

Models: Input contracts and adaptation

1 Model input contract

Expected representation, layout and semantic assumptions

2 RF data

Signal values, named axes and acquisition metadata

3 Explicit axis mapping

RF axes → Model input dimensions

Input adapter

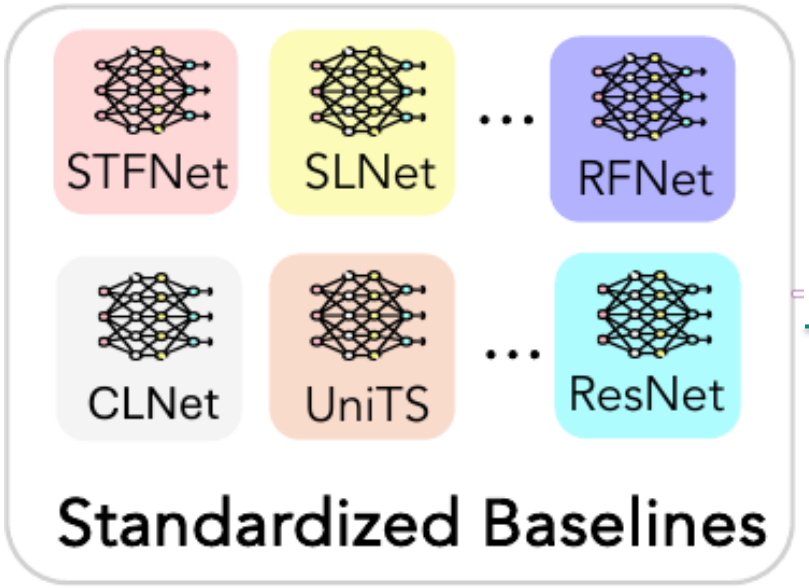
Apply the contract and axis mapping

Model-ready input

Representation + layout

Reusable input preparation for each model

Choose an existing baseline



Model interface

INPUT

Model-specific input requirements and adaptation

OUTPUT

Shared task interface

Training and evaluation

Reuse the workflow with the surrounding protocol held fixed.

Model example: Adapting data to RFNet


```
from octosense.transforms.adapters.auto_align import AutoAlign
```

1. Load the model and inspect its contract

```
model = octo.models.load(
    "rfnet", input_shape=(2560, 180), num_classes=6)
print(model.get_input_contract())
```

Model requirement

Real values · time × feature

2. Declare the named-axis mapping

```
align = AutoAlign(model_or_contract=model, axis_map={
    "time": "time", "feature": ("subc", "tx", "rx"),
})
```

Sample axes: time, subc, tx, rx

time → time
subc × tx × rx → feature
30 × 2 × 3 = 180

3. Apply the adapter

```
model_input = align(sample)
```

This example uses magnitude and the declared axis mapping.

Example input size: (2560, 30, 2, 3) Model batch layout: BTF

Experiments: Specifications and execution

DESIGN / 3. Specification-to-component mapping · STFNet example

```yaml
name: Widar3
task: gesture_classification
source: Intel_5300.dat
schema: WiFi CSI, complex64,
        [time,subc,tx,rx]

Normalize(axis=subc)
RxConjugate()
TemporalFilter(60 Hz)
ToBLC()

name: STFNet
input_layout: BLC
fixed_size: length=2560, channel=180
num_classes: 6

task: Widar3/gesture_classification
output: list[dict[index, score]]
```

Specification

Dataset & Dataloader
- Widar3 dataset instance
- RadioTensor samples
- Gesture labels and batching

Transform pipeline
- Subcarrier normalization
- Conjugate multiplication
- 60 Hz temporal filtering
- Layout conversion to BLC

Model instance
- Instantiated STFNet
- BLC input adapter
- 6-way classifier head

Runner / prediction contract
- Runner for gesture classification
- Prediction: index + score

Executable Pipeline

Specification → Pipeline: Build components

Pipeline → Specification: Export settings

Experiment example: Declare the configuration


```
from octosense.specs.schemas.benchmark import BenchmarkSpec
```

ONE CONFIGURATION / Read left, then continue on the right

Dataset + task

```
spec = BenchmarkSpec.from_dict({
 "dataset": {
  "dataset_id": "widar3",
  "modalities": ["wifi"],
  "variant": "g6",
  "split_scheme": "official"
 },
 "task": {
  "task_id":
   "classification/gesture",
  "task_binding": "gesture"
 },
```

Transform + model + runtime

```
 "transform": {"steps": []},
 "model": {
  "model_id": "rfnet",
  "entry_overrides": {
   "num_classes": 6
  }
 },
 "runtime": {
  "device": "cpu", "batch_size": 32,
  "epochs": 5, "seed": 41
 }
})
```

RFNet example 1/3: Declare the experiment. Input mapping follows on the next page.

Experiment example: Build, run and inspect

Use the Widar3 view and specification from the previous examples.

```
pipe = octo.pipelines.load(spec=spec, dataset=view)
pipe.auto_align({
   "time": "time", "feature": "subc*tx*rx",
})
print(pipe.describe_tree())
```

1 Assemble

Data + adapter
Model + training

```
run = pipe.train(
   output_root="runs", run_name="widar3-rfnet")
```

2 Run

Outputs +
spec snapshot

```
metrics = octo.benchmarks.evaluate(run)
```

3 Read results

Run metrics

RFNet example 2/3: Assemble the workflow, execute it and read the completed results.

Experiment example: Save and restore


```
from octosense.specs.serde.yaml_io import dump_yaml, load_yaml
```

```
spec_path = Path("widar3.yaml")
spec_path.write_text(
   dump_yaml(pipe.benchmark_spec), encoding="utf-8")
```

1 Export

Pipeline → YAML

```
restored_spec = load_yaml(spec_path)
restored_pipe = octo.pipelines.load(
   spec=restored_spec, dataset=view)
```

2 Restore

YAML → Pipeline

```
manifest = run["artifacts"]["run_manifest"]
saved_metrics = octo.benchmarks.evaluate(manifest)
```

3 Inspect

Manifest → Results

RFNet example 3/3: Save the configuration, rebuild the pipeline and inspect run results.

Evaluation: Three design outcomes

Engineering effort

How much pipeline code does the user still need to write?

Numerical behavior

Do matched experiments retain their original training behavior?

Component-assisted reproduction

Can the same coding agent recover the reference workflow?

EVALUATION / 1. Less repeated engineering

~87% fewer user-side function-call statements

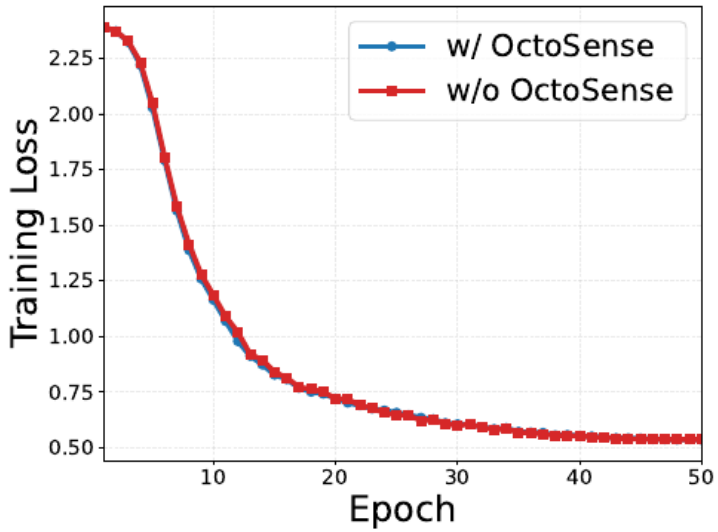
Dataset	Model	w/o OctoSense	w/ OctoSense	Reduction
Widar3.0	**STFNet**	**159**	**16**	**89.94%**
Widar3.0	RFNet	122	15	87.70%
Widar3.0	UniTS	150	15	90.00%
WiFi Gait	STFNet	124	13	89.52%
WiFi Gait	RFNet	87	12	86.21%
WiFi Gait	UniTS	115	12	89.57%
MM-Fi	ResNet18	118	11	90.68%
XRF55	ResNet18	114	20	82.46%
OctoNet	ResNet18	96	10	89.58%
HuPR	ResNet18	122	12	90.16%
HuPR	STFNet	115	23	80.00%
HuPR	RFNet	115	23	80.00%
HuPR	UniTS	115	23	80.00%

13 paired configurations · 80–91% reduction · LLOC counts user-side function-call statements

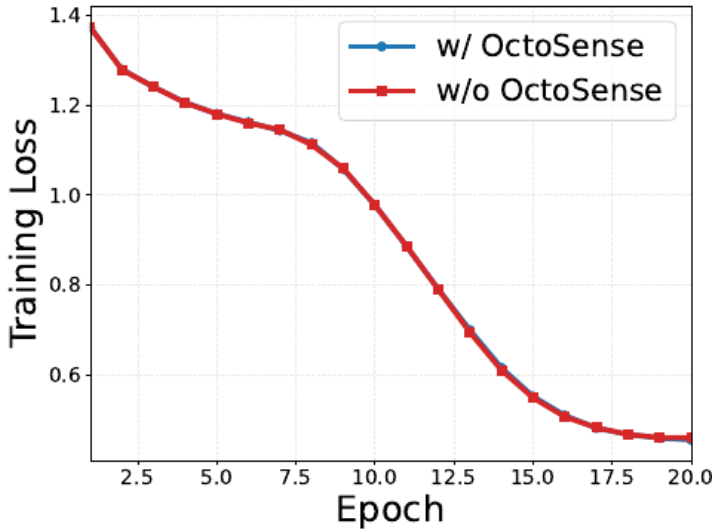
Evaluation: Numerical consistency

EVALUATION / 2. Preserved numerical behavior

Same data splits, architectures, training schedules and evaluation protocols



(a) STFNet × WiFi Gait



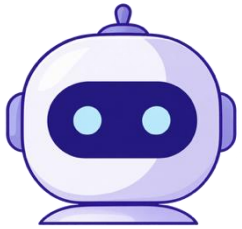
(b) RFNet × WiFi Gait

Paired configuration	Training loss	Test accuracy gap
WiFi Gait + STFNet	Closely aligned	At most 0.8 pp
WiFi Gait + RFNet	Closely aligned	At most 0.8 pp

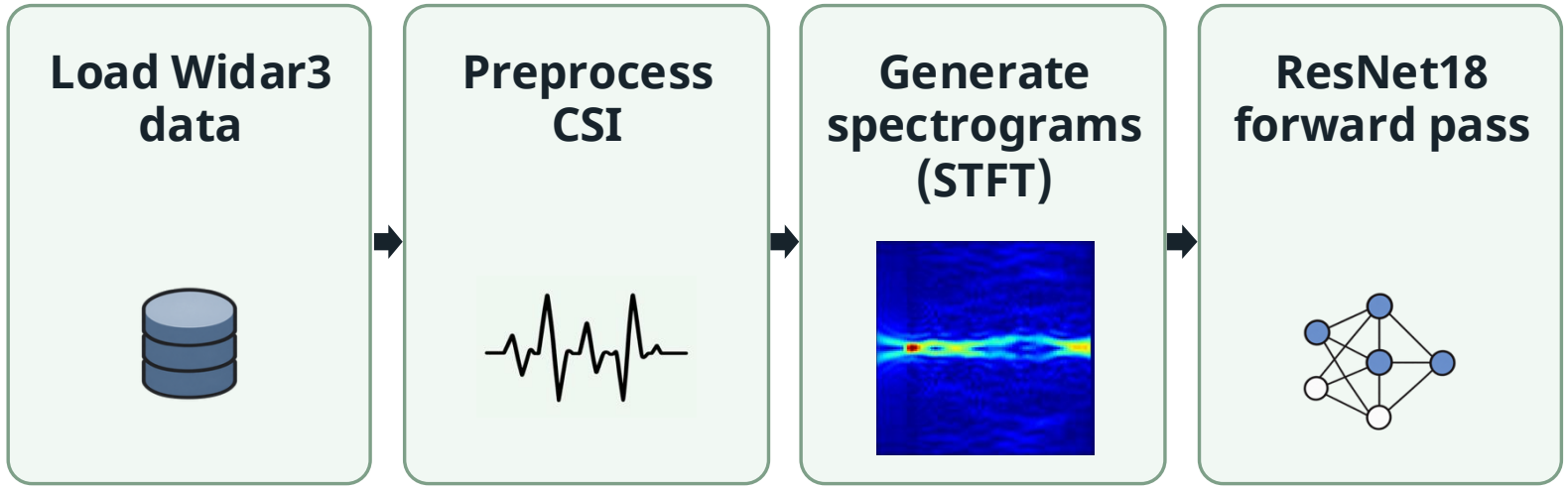
Evaluation: LLM-assisted reproduction

Task: reproduce a Widar3 sensing pipeline

GPT-5.3-codex



Writes code, runs tests, and fixes errors



We check output shapes and values against the same reference

Same task and test budget	Without OctoSense	With OctoSense
RF processing code	Implement from scratch	Reuse validated operators
Reference-output check	Failed after 15 tests	Passed on test 1
Tokens consumed	4.95M	1.94M (61% fewer)

One autonomous case study, with at most 15 test calls per setting

Takeaway

1 Shared RF data access

Signal I/O with named axes; metadata views for selection.

2 Composable processing and model interfaces

Operator contracts, model inputs and explicit axis mappings.

3 Executable, reusable experiments

Pipelines run the workflow; specs record and restore it.

Paper

GitHub

UbiComp ISWC 2026

香 港 大 學
THE UNIVERSITY OF HONG KONG

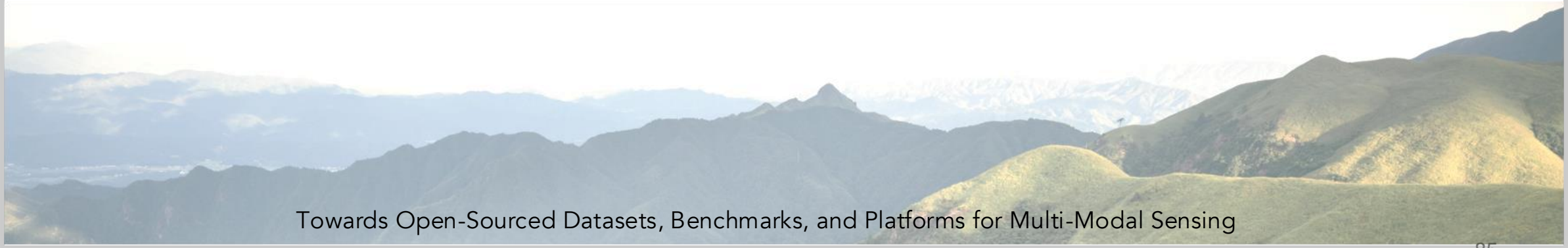
HKU AIoT LAB 香港大學人工智能物联网實驗室

MODULE II

Models and learning paradigms

Speakers: Xie Zhang, Luca Jiang-tao Yu

What kind of model should sit on the multi-modal sensing data?

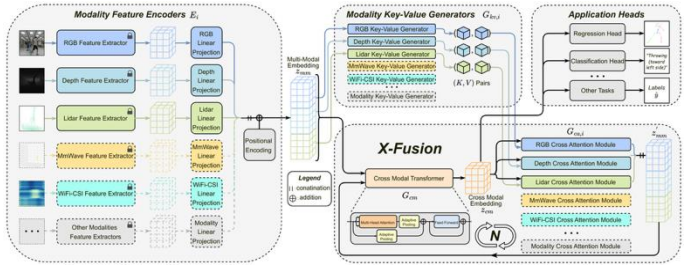


A brief review on multi-modal sensing models

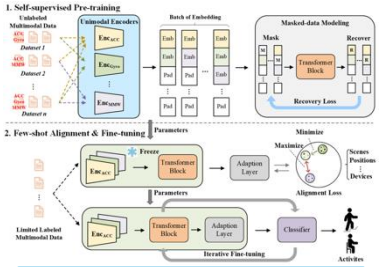
❖ Fuse sensing modalities into one embedding

- One **encoder** per modality
- **Cross-attention** to fuse those features

(e.g., X-Fi, MASTER)



X-Fi [ICLR'25]

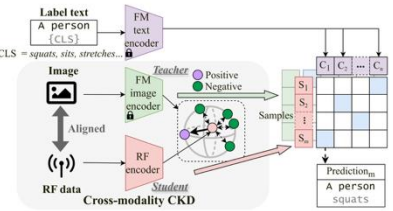


MASTER [IMWUT'25]

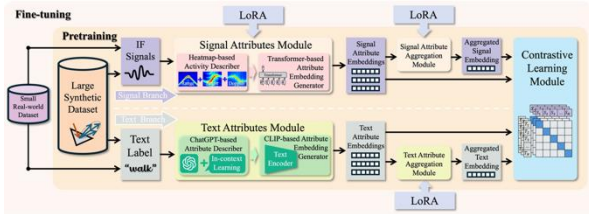
❖ Align sensing data with language or vision

- Language: Encode the **text label** with a pretrained language foundation model, Then, align the sensing embedding with that **language embedding**.
- Vision: Encode the **paired image** with a pretrained vision foundation model, Then, align the sensing embedding with that **vision embedding**.

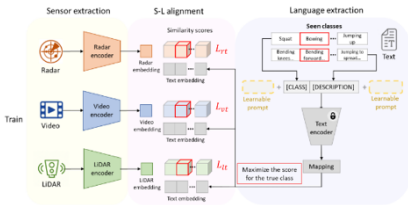
(e.g., FM-Fi, mmCLIP, TENT)



FM-Fi [SenSys'24]



mmCLIP [SenSys'24]

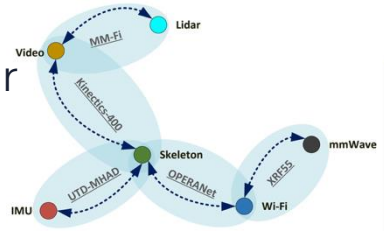


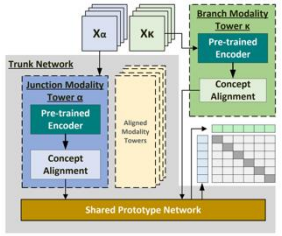
TENT [TMC'26]

❖ Align sensing modalities with each other

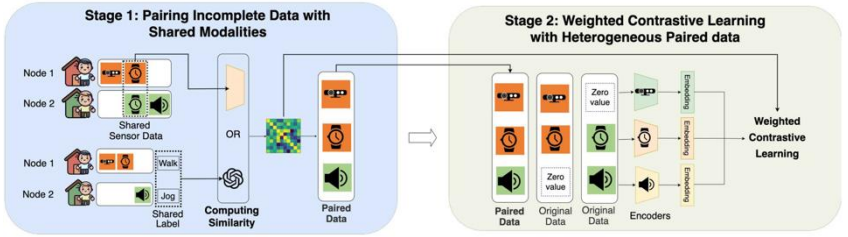
- **Paired** sensing modalities
- **Contrastive learning** aligns their embeddings

(e.g., BABEL, MMBind)



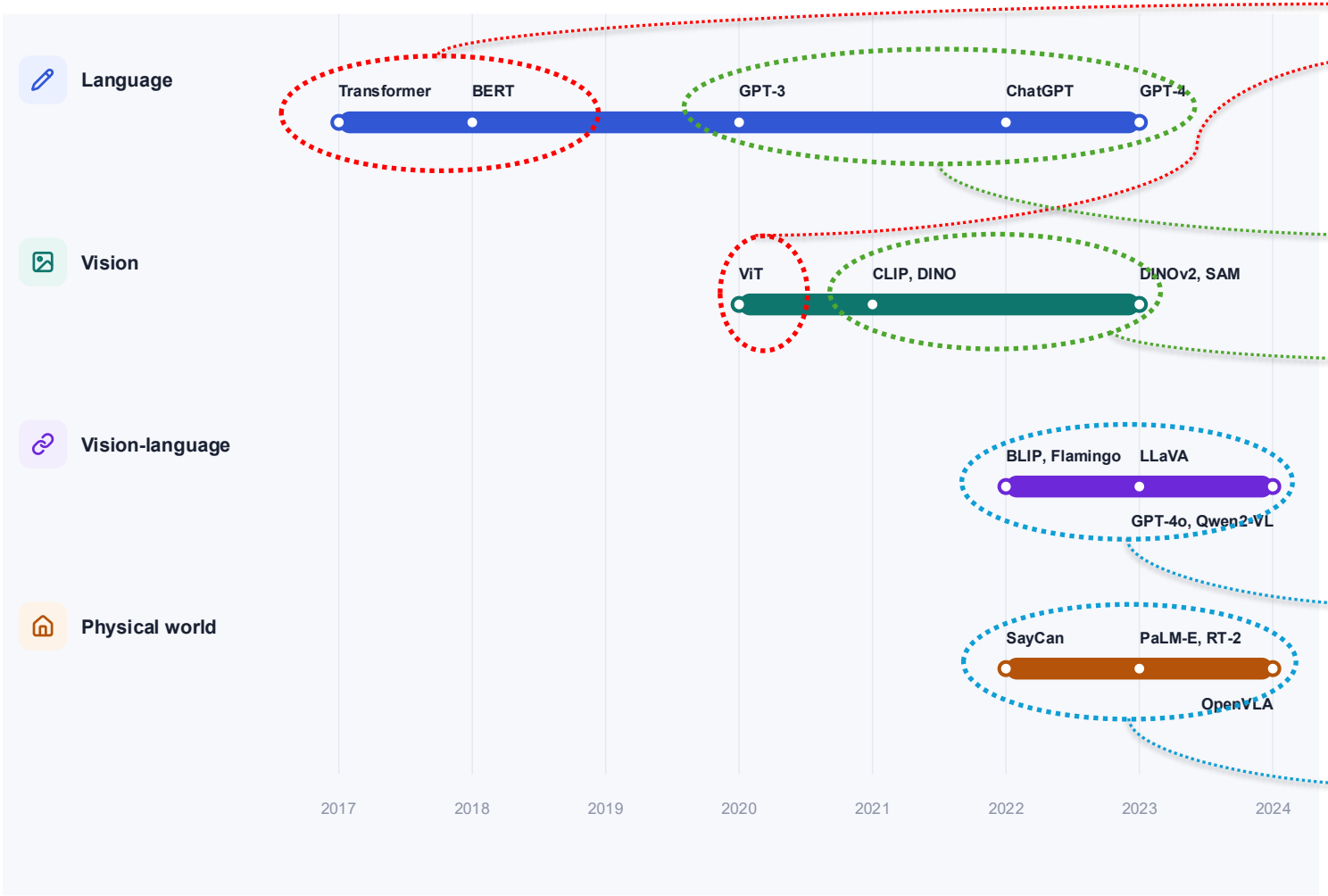


BABEL [SenSys'25]



MMBind [SenSys'25]

A brief history of vision–language models



❖ One backbone: the Transformer

- **Language** takes it up first.
- **Vision:** ViT turns an image into tokens.

❖ Foundation models

- Language foundation model: **scaling with next token prediction**
- Vision.
  - **Align to language**: CLIP.
  - **Self-supervised Learning** on images alone: DINO.

❖ Vision–language models

VLM: Connect the **vision foundation model generated vision embedding** to a language model, so one model takes image and text.

VLA: Keep the joint embedding. Change the output **from text to an action token**.

Where the analogy breaks

01 The signal is physical

A pixel records appearance. A phrase encodes a concept. Wi-Fi, radar, thermal, and IMU observe the world through different physical principles.

- Is the Transformer the right backbone for these modalities, or how should it be adapted to each sensor's physics?
- How do we combine the generalizability of physics with the fitting power of deep learning?
- How should proven signal-processing methods enter a data-driven model?
- What are effective learning tasks and objectives for sensing foundation models?
- …….

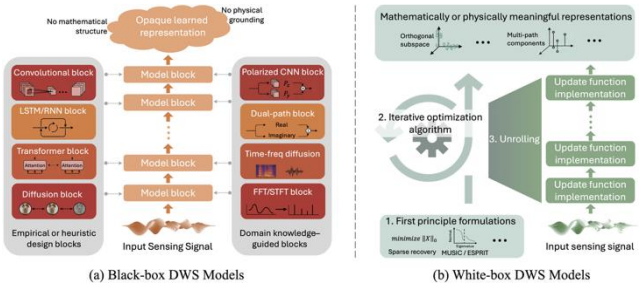
02 The sensing modalities are diverse

Sensing spans far more modalities than the vision–language setting.

- Is aligning a sensing embedding to a language or vision foundation embedding a good approach, as in CLIP-style methods?
- Is contrastive learning on paired sensing modalities a good path to a unified representation, as in Babel?
- Should each modality have its own foundation model, or should many modalities share one?
- Is there a natural center modality to build first, then align the others to it—as language came first, then CLIP aligned vision to it?
- ……..

Three dedicated designs

UbiComp'26



(a) Black-box DWS Models (b) White-box DWS Models

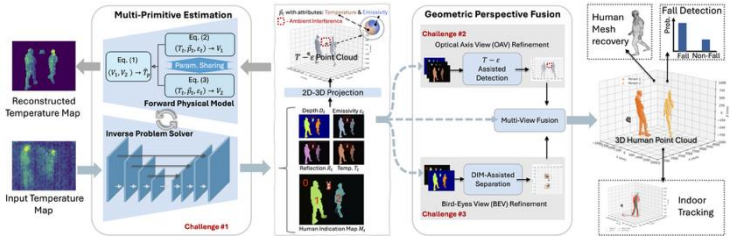
RF-CRATE

Speaker: Xie Zhang

White-box deep wireless sensing with fully mathematically derivable architectures across RF modalities.

- Is the Transformer the right backbone for these modalities, or how should it be adapted to each sensor's physics?

MobiCom'26



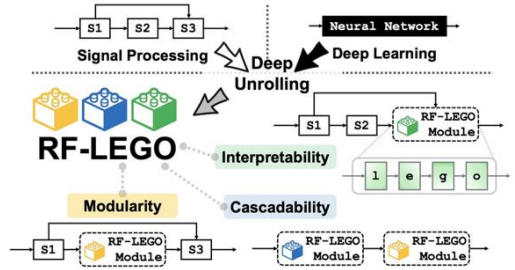
TAP3D

Speaker: Xie Zhang

Physics-informed learning for thermal-assisted 3D human point cloud generation.

- How do we combine the generalizability of physics with the fitting power of deep learning?

MobiCom'26



RF-LEGO

Speaker: Luca Jiang-tao Yu

Modular signal-processing and deep-learning co-design for RF sensing via deep unrolling.

- How should proven signal-processing methods enter a data-driven model?

Coffee Break

Will be back at 4:00 PM

Next:

Module II Models and learning paradigms

